

Plumb — System Architecture (AWS)

Complete Service Inventory, AWS Mapping, and Flow Diagrams

Engineering Team · Plumb

August 2026

Executive Summary

Table of Contents

1. System Overview

2. AWS Service Inventory

3. Network Topology

3.1 Subnet design rationale

3.2 Security groups (per service)

4. Service Inventory

4.1 Phase 0 services (built; running on EKS)

4.2 Phase 1 services (specification)

4.3 Phase 2 services

4.4 Shared infrastructure services

5. Flow Diagrams

5.1 User opens the dashboard

5.2 Field user starts a 360° capture

5.3 Capture upload — chunk-by-chunk

5.4 AI Copilot query

5.5 Multi-stakeholder live walkthrough

5.6 Admin runs a SOC 2 audit query

6. Data Architecture

6.1 Logical data model

6.2 S3 bucket layout

6.3 TimescaleDB (capture telemetry)

6.4 ClickHouse (analytics)

6.5 Redis usage

7. Compute Architecture

7.1 EKS cluster topology

- 7.2 Node groups
- 7.3 Deploy topology per service
- 7.4 Ingress
- 7.5 Step Functions (capture pipeline)
- 8. Networking & Service Mesh
 - 8.1 Service mesh: App Mesh vs Istio (decision)
 - 8.2 Service-to-service auth (Phase 2+)
 - 8.3 Inter-AZ traffic
 - 8.4 Inter-region traffic
- 9. Identity & Access
 - 9.1 User identity (Cognito + external IdP)
 - 9.2 Workload identity (IAM Roles for Service Accounts)
 - 9.3 Tenant isolation
- 10. Observability
 - 10.1 The three pillars
 - 10.2 SLO definitions
 - 10.3 Alert routing
 - 10.4 Cost observability
- 11. Security & Compliance
 - 11.1 Layered defenses
 - 11.2 Compliance posture
 - 11.3 FedRAMP boundary (us-gov-east-1)
- 12. Multi-Region & Disaster Recovery
 - 12.1 Region topology
 - 12.2 RPO / RTO targets
 - 12.3 Active-active vs active-passive
 - 12.4 DR drill cadence
- 13. Cost Model
 - 13.1 Variable cost per capture
 - 13.2 Per-tenant monthly cost (Pro tier)
 - 13.3 Reserved capacity to plan for
- 14. Deployment Topology
 - 14.1 CI/CD pipeline
 - 14.2 Argo Rollouts example (Phase 0 CI → Production)
 - 14.3 IaC layout
 - 14.4 Promotion flow
 - 14.5 Secrets management
- 15. Risks & Trade-offs
- 16. Open Decisions
- Closing Notes

Executive Summary

This document is the **system architecture** for Plumb — the AWS deployment target that complements the Phase 0 report. While the Phase 0 report documents *what* was built (the foundation), this document specifies *how* the full 13-product platform runs in production on AWS.

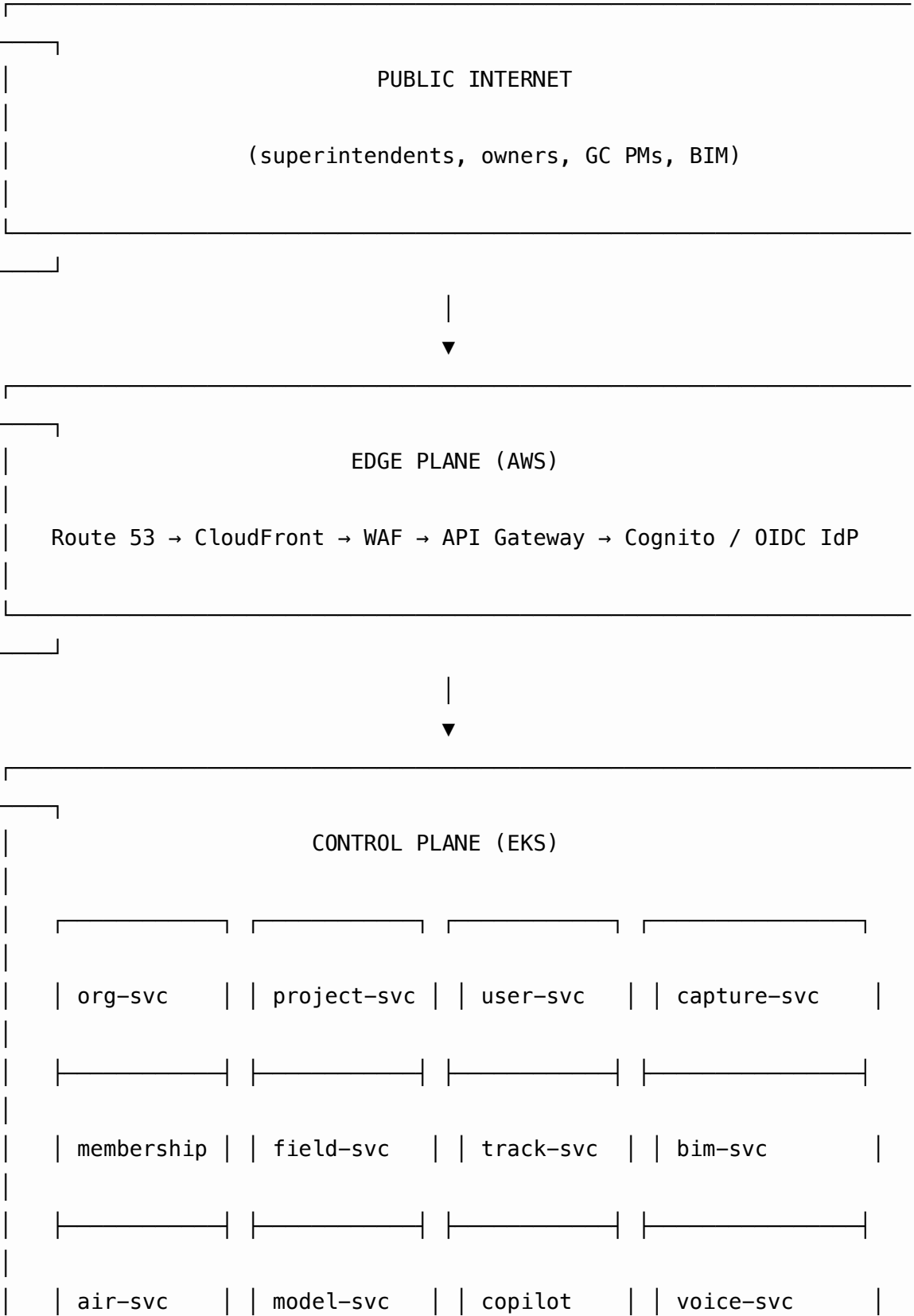
The architecture has three planes — **edge, control, data** — and three layers of trust — **public internet, authenticated API surface, internal service mesh**. Every service is deployed as a container on EKS, fronted by an Envoy-based API gateway, with RDS Postgres for the system of record, S3 for media, ElastiCache Redis for sessions and cache, and a dedicated GPU node group for the ML/3D pipeline.

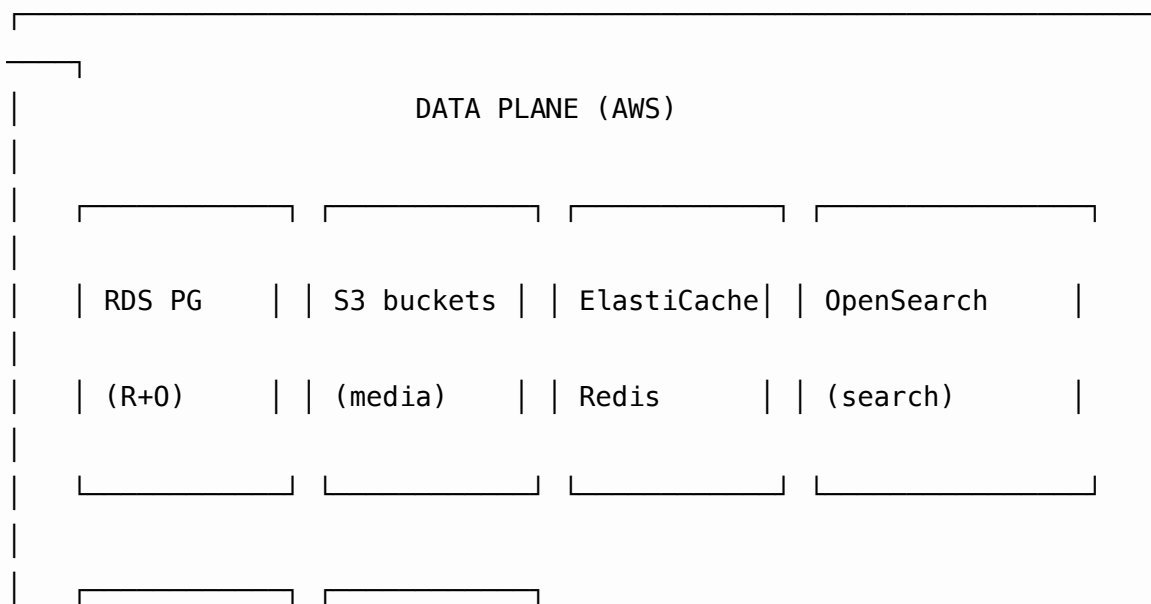
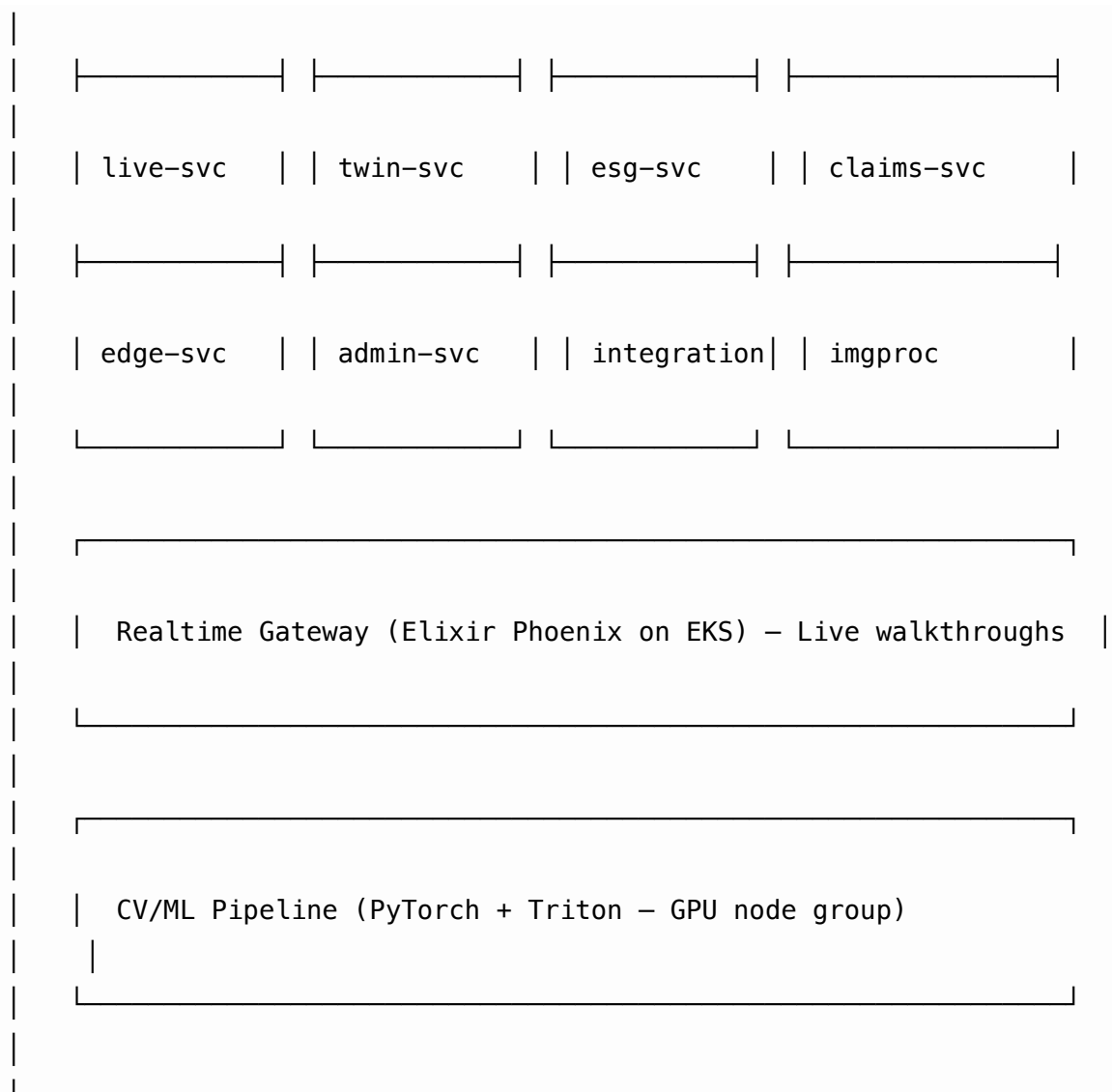
Table of Contents

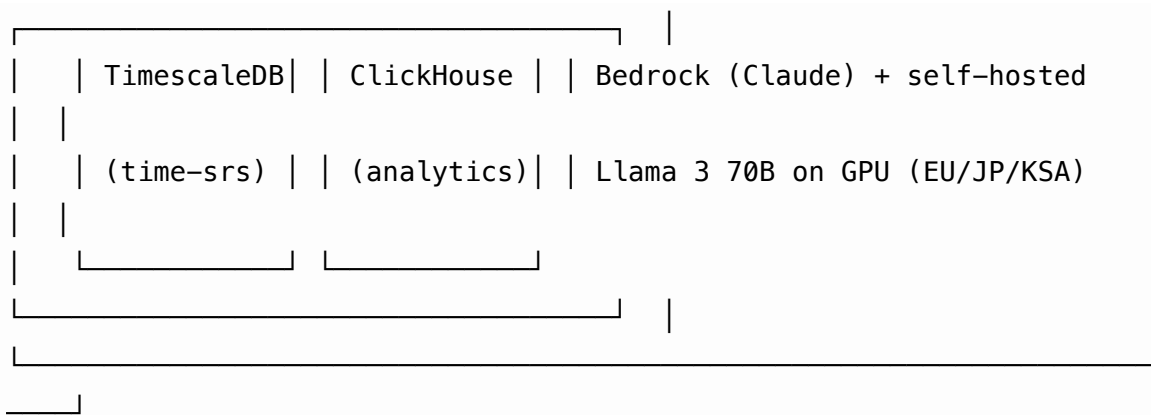
1. **System Overview** — 30,000-foot view
2. **AWS Service Inventory** — every AWS service mapped to a Plumb component
3. **Network Topology** — VPCs, subnets, security groups, transit gateway
4. **Service Inventory** — every Plumb service with its AWS deployment
5. **Flow Diagrams** — the major request paths
 - 5.1 User opens the dashboard
 - 5.2 Field user starts a 360° capture
 - 5.3 Capture upload
 - 5.4 AI Copilot query
 - 5.5 Multi-stakeholder live walkthrough
 - 5.6 Admin runs a SOC 2 audit query
6. **Data Architecture** — Postgres schemas, S3 buckets, Redis usage
7. **Compute Architecture** — EKS clusters, GPU nodes, autoscaling
8. **Networking & Service Mesh** — Envoy, SPIFFE, mTLS
9. **Identity & Access** — Cognito, IAM, OIDC, SAML
10. **Observability** — OpenTelemetry, CloudWatch, X-Ray, Grafana
11. **Security & Compliance** — KMS, HSM, FedRAMP boundary, WAF
12. **Multi-Region & Disaster Recovery** — active/active, RPO/RTO
13. **Cost Model** — per-tenant unit economics on AWS
14. **Deployment Topology** — IaC, GitOps, and progressive delivery

1. System Overview

Plumb is a multi-tenant SaaS platform for construction visual intelligence. Three planes:







2. AWS Service Inventory

Every AWS service used by Plumb, with the Plumb component it serves and the closest non-AWS alternative annotated for clarity.

Layer	AWS Service	Plumb Component	Notes
DNS	Route 53	Apex + per-region failover	Health-checked alias records to CloudFront
CDN	CloudFront	Static assets, marketing site, 360° tile streaming	OAI to S3; Lambda@Edge for auth checks
WAF	AWS WAF	Edge protection	OWASP Top 10 managed rules + custom rate-limit rules
DDoS	AWS Shield Standard + Advanced	Edge protection	Automatic
API Gateway	AWS API Gateway (REST)	Public REST surface	VPC integration; integrates with WAF
Load Balancer	Network Load Balancer (NLB)	Internal service ingress	TLS termination at Ingress-NGINX

Layer	AWS Service	Plumb Component	Notes
Service Mesh	App Mesh (Envoy)	East-west traffic	mTLS via SPIFFE; 2.5% sampling to X-Ray
Compute	EKS (Kubernetes)	All services	Karpenter for node autoscaling
GPU	EC2 P5 / G5 instances	CV/ML pipeline, 3DGS, ME	Karpenter MIG profiles for fine-grained scheduling
Serverless	Lambda	OIDC callbacks, S3 event triggers, ConMon	1-2 GB / 30s timeout
Database	RDS for PostgreSQL (Aurora)	System of record	Multi-AZ, encryption at rest, KMS-managed
Time-series	Timescale Cloud (managed) on RDS	Capture telemetry, sync events	Hypertable partitioning
OLAP	ClickHouse Cloud (or Redshift Serverless)	Analytics dashboards	100x cheaper than Postgres for scans
Search	OpenSearch Service	Federated search	Per-tenant index aliases
Vector	Aurora pgvector (Phase 1) → OpenSearch k-NN (Phase 2)	Copilot RAG	Hybrid BM25 + dense
Cache	ElastiCache for Redis	Sessions, rate-limit, idempotency, presence	Cluster mode, AOF persistence
Queue	SQS + Kinesis Data Streams	Ingestion, telemetry, webhooks	Standard + FIFO; Kinesis for backpressure

Layer	AWS Service	Plumb Component	Notes
Object storage	S3	Media (raw 360, frames, tiles, BIM, drone)	Lifecycle: hot → IA → Glacier
Block storage	EBS gp3	Postgres data, Redis	Encrypted, gp3 for cost
CDN origin	S3 + CloudFront Origin Shield	Tile streaming	Per-tenant signed URLs
Secrets	AWS Secrets Manager	DB creds, OIDC client secrets	Auto-rotation 90d
KMS	AWS KMS + CloudHSM	All encryption-at-rest	HSM-backed root for FedRAMP-High
IAM	AWS IAM + IRSA	Service-to-AWS auth	SPIFFE/SVID → IAM roles via OIDC
Identity	Amazon Cognito (user pools) + Auth0 / Okta	Customer IDP	Federated SSO per tenant
Logging	CloudWatch Logs + S3	Structured logs	OTel → Firehose → S3 + OpenSearch
Metrics	CloudWatch Metrics + Prometheus	Service metrics	1s resolution for control-plane alerts
Tracing	AWS X-Ray + OpenTelemetry	Distributed tracing	2.5% sampling default
Dashboards	CloudWatch Dashboards + Grafana	Ops + product dashboards	Grafana for SLOs
Alerts	CloudWatch Alarms + PagerDuty	On-call paging	SEV1-4 severity model
Secrets	AWS Secrets Manager	Cross-service secrets	Per-secret IAM policy

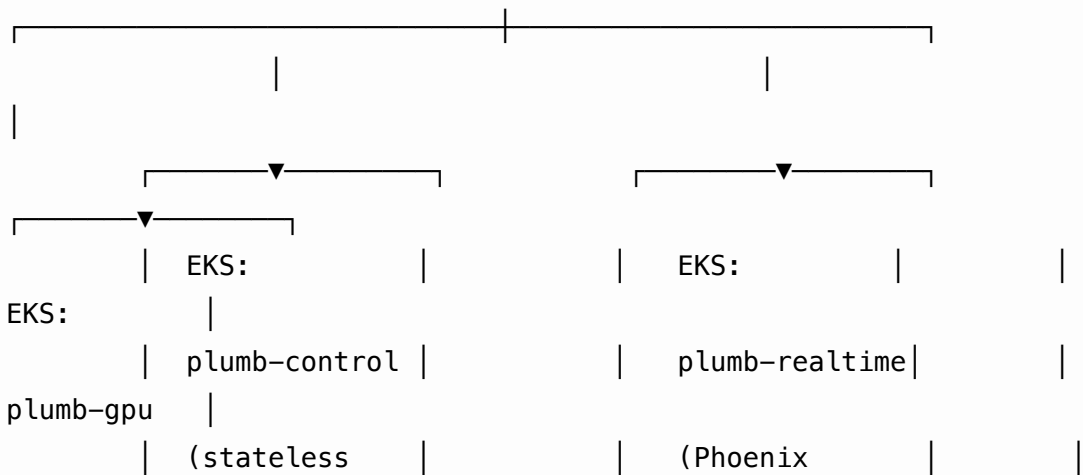
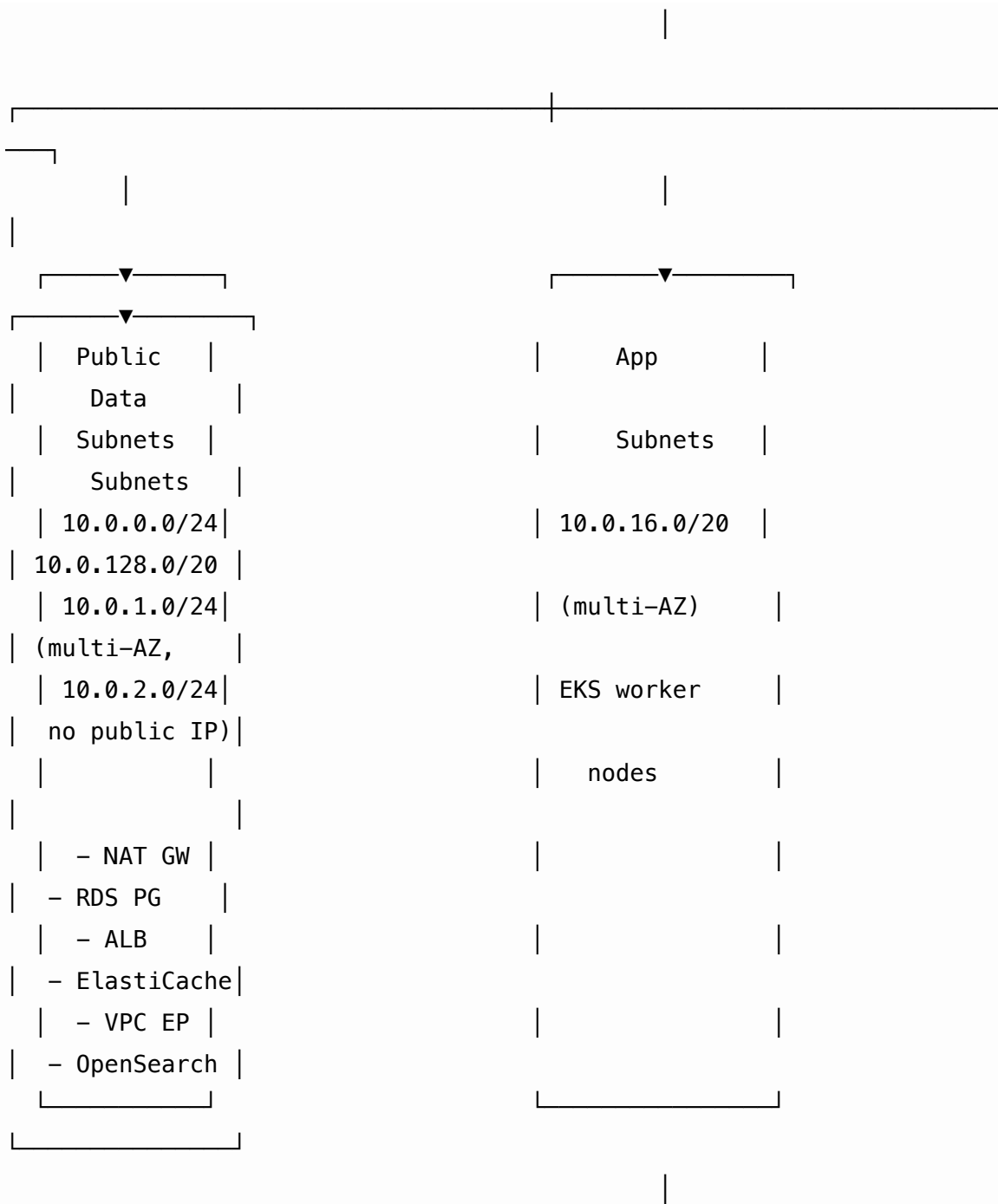
Layer	AWS Service	Plumb Component	Notes
Containers	ECR	Service images	Lifecycle policies: keep last 30
CI/CD	GitHub Actions + ArgoCD	Build + deploy	GitOps on every commit
IaC	Terraform + AWS CDK	Infrastructure	Per-region state files in S3
Key mgmt	AWS KMS + CloudHSM	CMK per tenant, per-region	HSM-backed root for FedRAMP-High
WAF	AWS WAF	Layer 7 protection	Managed rule sets + custom
DNS	Route 53	Geo-routing for FedRAMP-boundary	Latency-based records
Cost	AWS Cost Explorer + CUR	Per-tenant cost attribution	Tag-based allocation
DR	AWS Backup + S3 Cross-Region Replication	RPO $\leq$ 15 min / RTO $\leq$ 4 h	Daily drills
Audit	AWS CloudTrail + S3 Object Lock	Compliance evidence	7-year retention
Secrets	AWS Secrets Manager	Per-tenant credentials	Cross-region replication

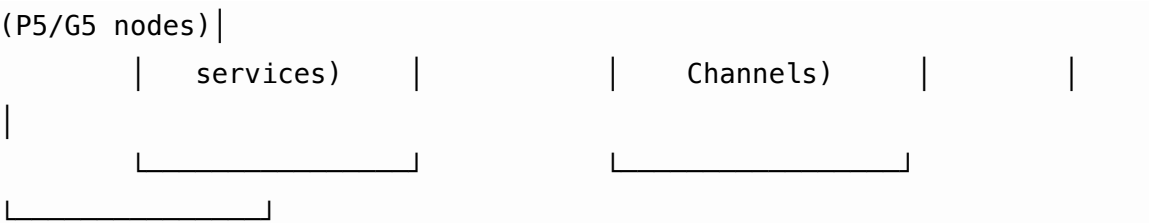
3. Network Topology

Each AWS region deploys the same VPC layout. The diagram below is for us-east-1; eu-west-1, ap-southeast-2, ap-northeast-1, and ksa-central-1 mirror it with regional IP ranges.

```
AWS Region: us-east-1
```

```
VPC: 10.0.0.0/16 (plumb-prod-use1)
```



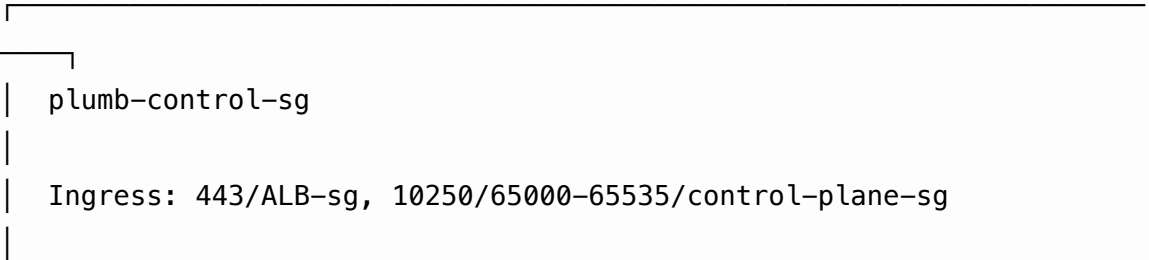


- INTER-REGION:
- Transit Gateway peering: us-east-1 ↔ us-west-2 ↔ eu-west-1
 - GovCloud (us-gov-east-1) is ISOLATED – no transit peering to commercial
 - S3 Cross-Region Replication: raw-360 → eu-west-1 (active-active)
 - RDS Cross-Region Read Replica: us-east-1 → us-west-2 (warm standby)
 - Route 53 latency-based routing across regions

3.1 Subnet design rationale

Subnet tier	CIDR	Purpose	Ingress
Public	/24 × 3 AZs	NAT gateway, ALB, VPC endpoints	Internet Gateway
App	/20 × 3 AZs	EKS worker nodes (stateless services)	NAT → Internet for outbound
Data	/20 × 3 AZs	RDS Aurora, ElastiCache, OpenSearch	No internet; VPC endpoints only
GPU	/20 × 3 AZs	EKS GPU node group (P5/G5)	App subnets + EFA
Spare	—	Reserved for VPC expansion	Future

3.2 Security groups (per service)



Egress: 5432/data-sg, 6379/redis-sg, 443/0.0.0.0/0
plumb-realtime-sg (Phoenix Channels) Ingress: 443/ALB-sg, 4000-4010 (WebSocket) Egress: 6379/redis-sg
plumb-gpu-sg (CV/ML pipeline) Ingress: 10250/control-plane-sg Egress: 5432/data-sg, 443/S3-vpce, 443/Bedrock-vpce
plumb-data-sg (RDS, ElastiCache, OpenSearch) – NO INGRESS Ingress: 5432/control-sg, 5432/gpu-sg, 6379/control-sg, 9200/control

4. Service Inventory

Every Plumb service, its AWS deployment shape, and the AWS services it depends on. **Phase 0 services** are built; **Phase 1+ services** are specified in the Phase 0 report.

4.1 Phase 0 services (built; running on EKS)

Service	Image	EKS Cluster	Replicas (min/max)	Dependencies
org-service	org-service:v0.1.0	plumb-control	2/6	RDS Postgres, Secrets Manager
project-service	project-service:v0.1.0	plumb-control	2/6	RDS Postgres
user-service	user-service:v0.1.0	plumb-control	2/6	RDS Postgres, Amazon Cognito
membership-service	membership-service:v0.1.0	plumb-control	2/6	RDS Postgres, user-service
dashboard (Next.js)	dashboard:v0.1.0	plumb-control	2/10	EFS for .next/cache, all services

4.2 Phase 1 services (specification)

Service	Purpose	AWS Resources	GPU?
copilot-service	LLM gateway, function-calling, RAG	EKS control plane + Bedrock (Claude) + Aurora pgvector	No
capture-service	360° video ingestion, upload-session orchestration	EKS control plane + S3 + SQS + Step Functions	No
imgproc-service	Spatial AI: COLMAP/GLOMAP, OpenMVS, 3D	EKS GPU node group	Yes — P5/H100

Service	Purpose	AWS Resources	GPU?
	Gaussian Splatting, SAM-2, DINOv2	(P5.48xlarge) + FSx for Lustre	
bim-service	BIM ingest (IFC, Revit, NWD), tessellation, clash detection	EKS control plane + S3 + 3D Tiles hosting	No
field-service	Field notes, issues, dictation, sketches	EKS control plane + RDS Postgres + S3 (attach)	No
track-service	Progress tracking: BIM-vs-reality alignment, percent-complete	EKS control plane + TimescaleDB + ClickHouse	No
model-service	3D model serving, MeshLab glTF	EKS control plane + S3 + CloudFront	No
air-service	Drone coordination, LAANC, BVLOS-aware flight logs	EKS control plane + S3 + Kinesis (telemetry)	No
admin-service	Org/project management, billing, audit log	EKS control plane + RDS Postgres + Athena	No
integration-service (x14)	Connectors: Procore, ACC, BIM 360, P6, Slack, Teams, etc.	EKS control plane + SQS + Secrets Manager	No

4.3 Phase 2 services

Service	AWS Resources
voice-service	Lambda + Transcribe + Polly (realtime)
live-service	EKS Phoenix + IVS (real-time streaming) + Kinesis Video

Service	AWS Resources
twin-svc	EKS + IoT TwinMaker + S3
esg-svc	EKS + Bedrock (Anthropic) for report generation
claims-svc	EKS + QLDB (immutable ledger) + S3

4.4 Shared infrastructure services

Service	AWS Resource	Purpose
realtime-gateway	EKS Phoenix Channels on plumb-realtime cluster	WebSocket presence + live walkthroughs
pipeline-orchestrator	Step Functions + Lambda	Coordinates Spatial AI pipeline
tile-server	CloudFront + Lambda@Edge	Streams 3D tiles to browsers
auth-broker	EKS + Amazon Cognito + Lambda	OAuth2/OIDC + SAML
audit-collector	Kinesis Firehose → S3 + Athena	Compliance evidence
cost-collector	AWS Cost and Usage Reports → S3 → Athena	Per-tenant cost allocation
event-bus	Amazon EventBridge	Cross-service domain events

5. Flow Diagrams

The flow diagrams below show the **major request paths** through the system. Each diagram names the AWS service, the Plumb service, and the data exchanged at each step.

5.1 User opens the dashboard

A VDC PM opens `https://app.plumb.dev` and lands on the homepage showing org/project rollups.

US-EAST-1

Browser

CloudFront

API GW

Next.js

ALB

(REST)

Dashboard

1. GET /

(browser caches

via SW; first

load hits CF)

CloudFront smart-routing:

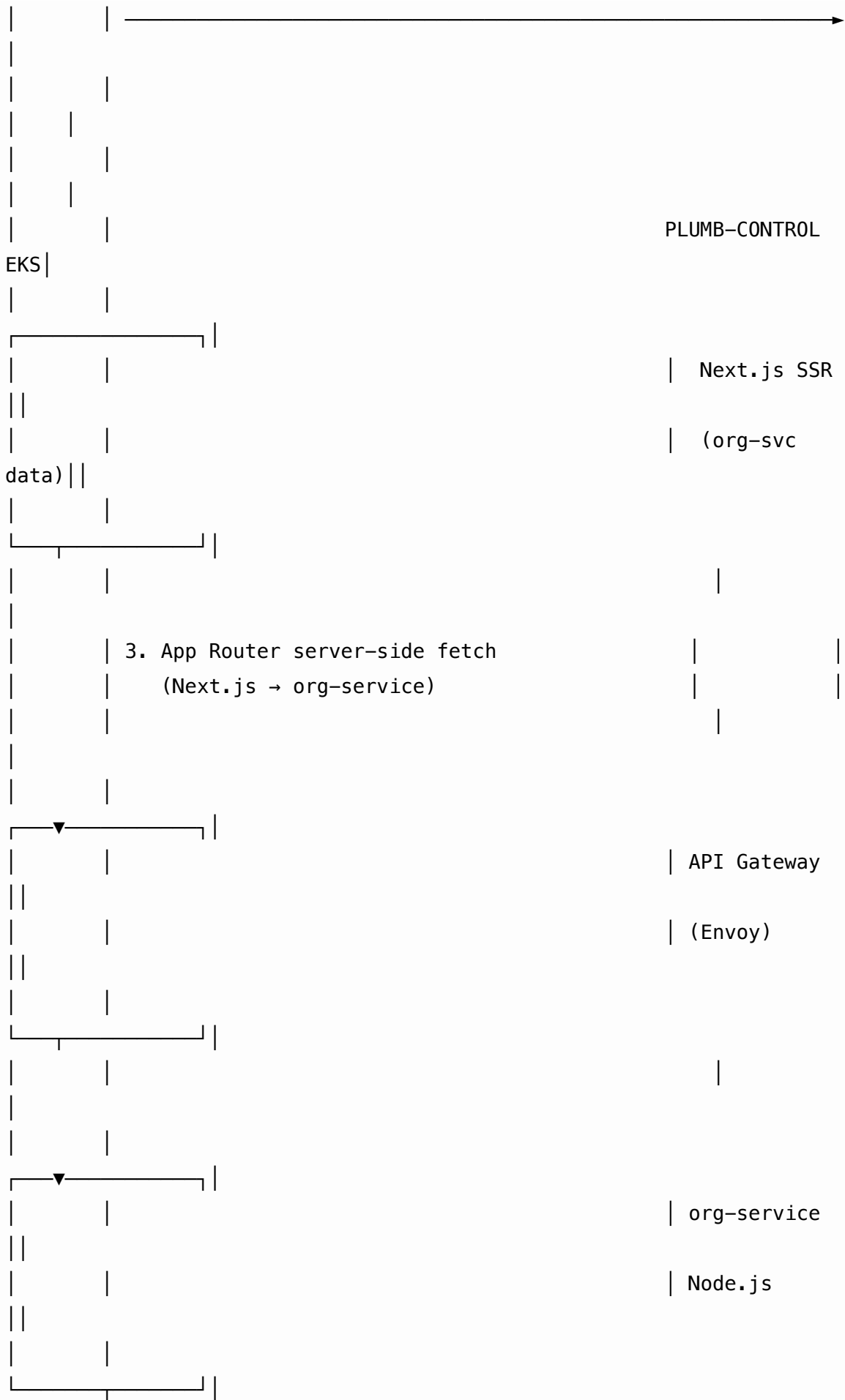
- /_next/static/* → S3 (cache hit)

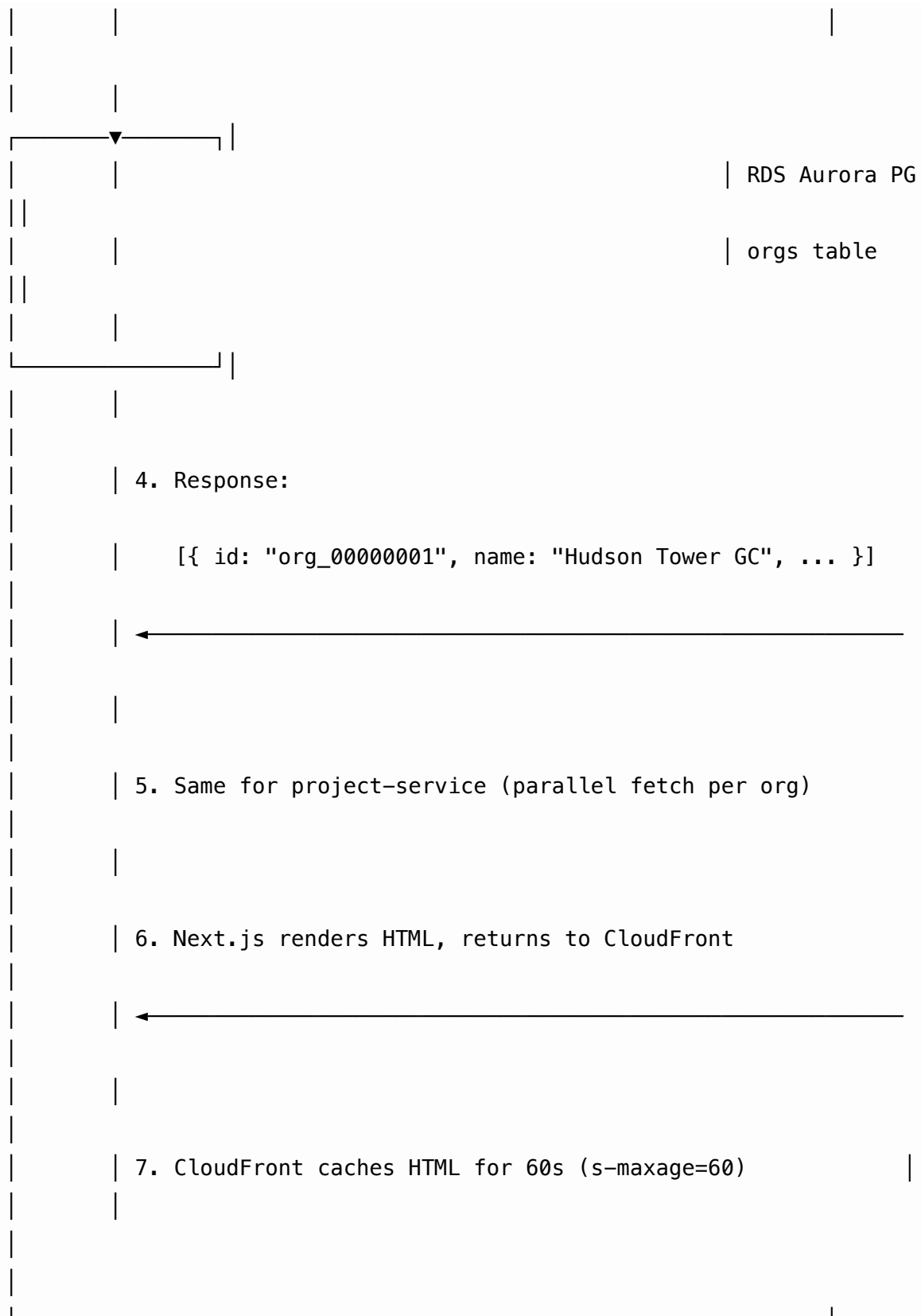
- /api/* → ALB → Next.js

2. GET /v1/orgs (SSR data fetch)

Authorization: Bearer eyJ... (Cognito JWT)

X-Request-Id: req_dashboard_abc123

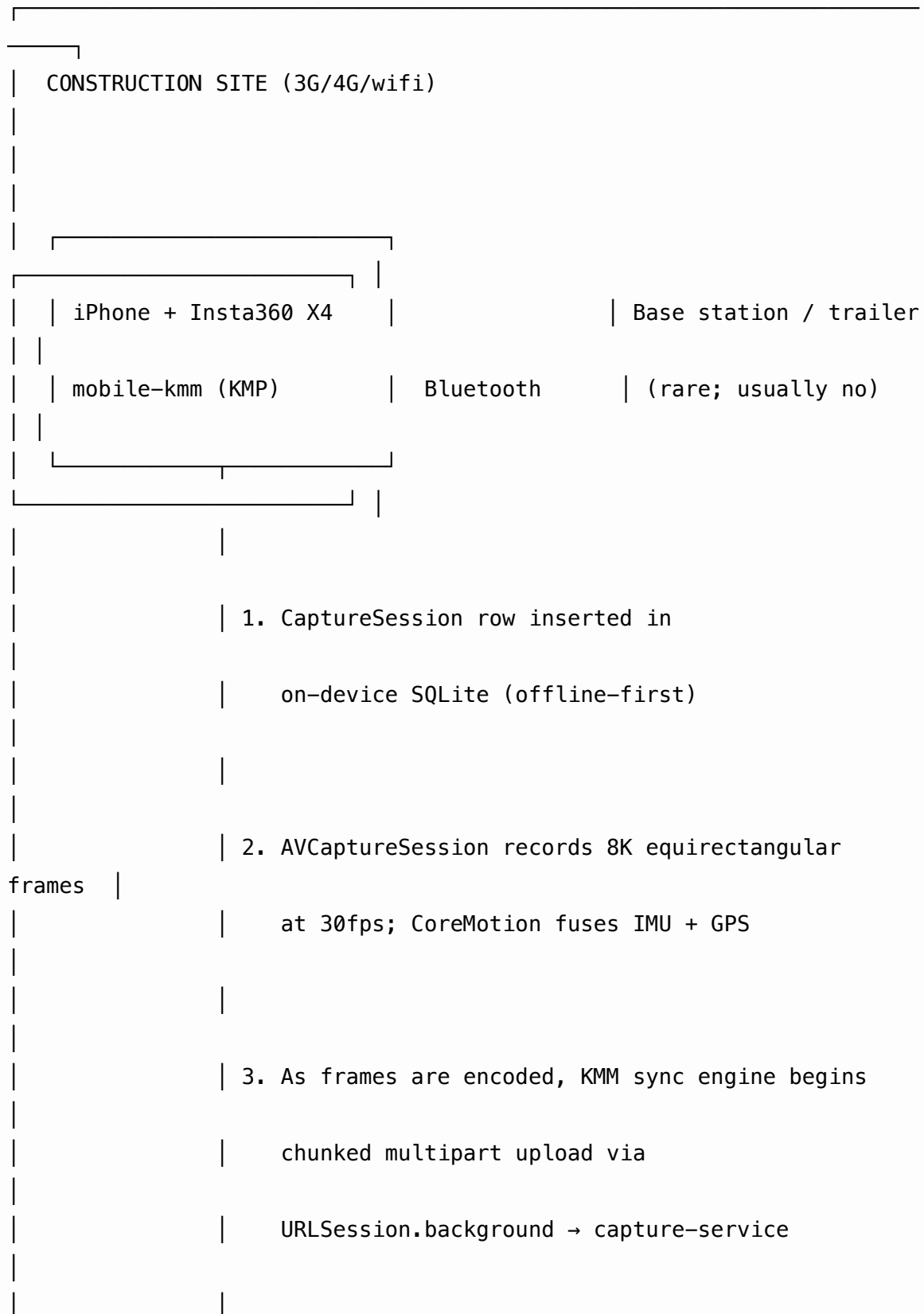


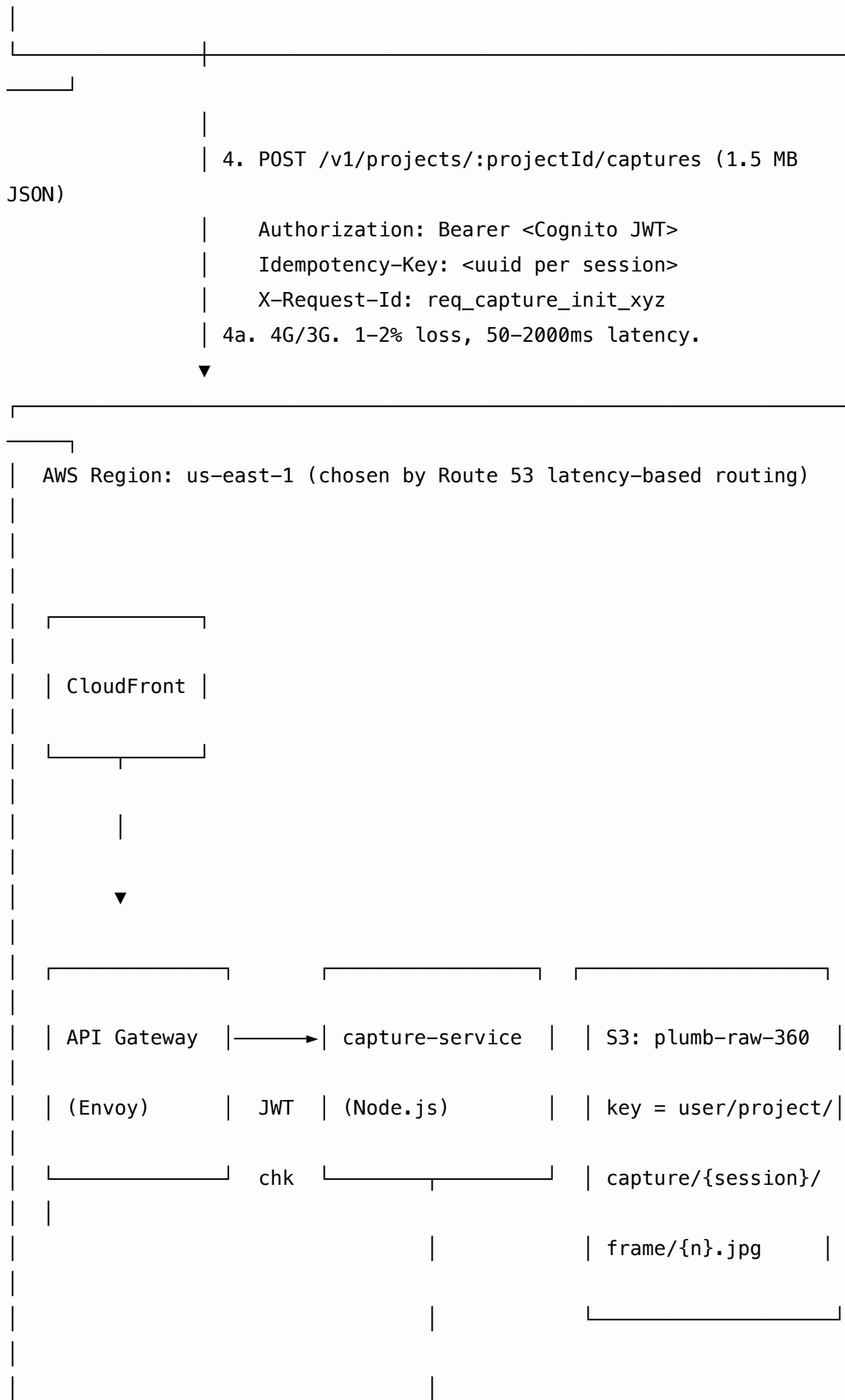


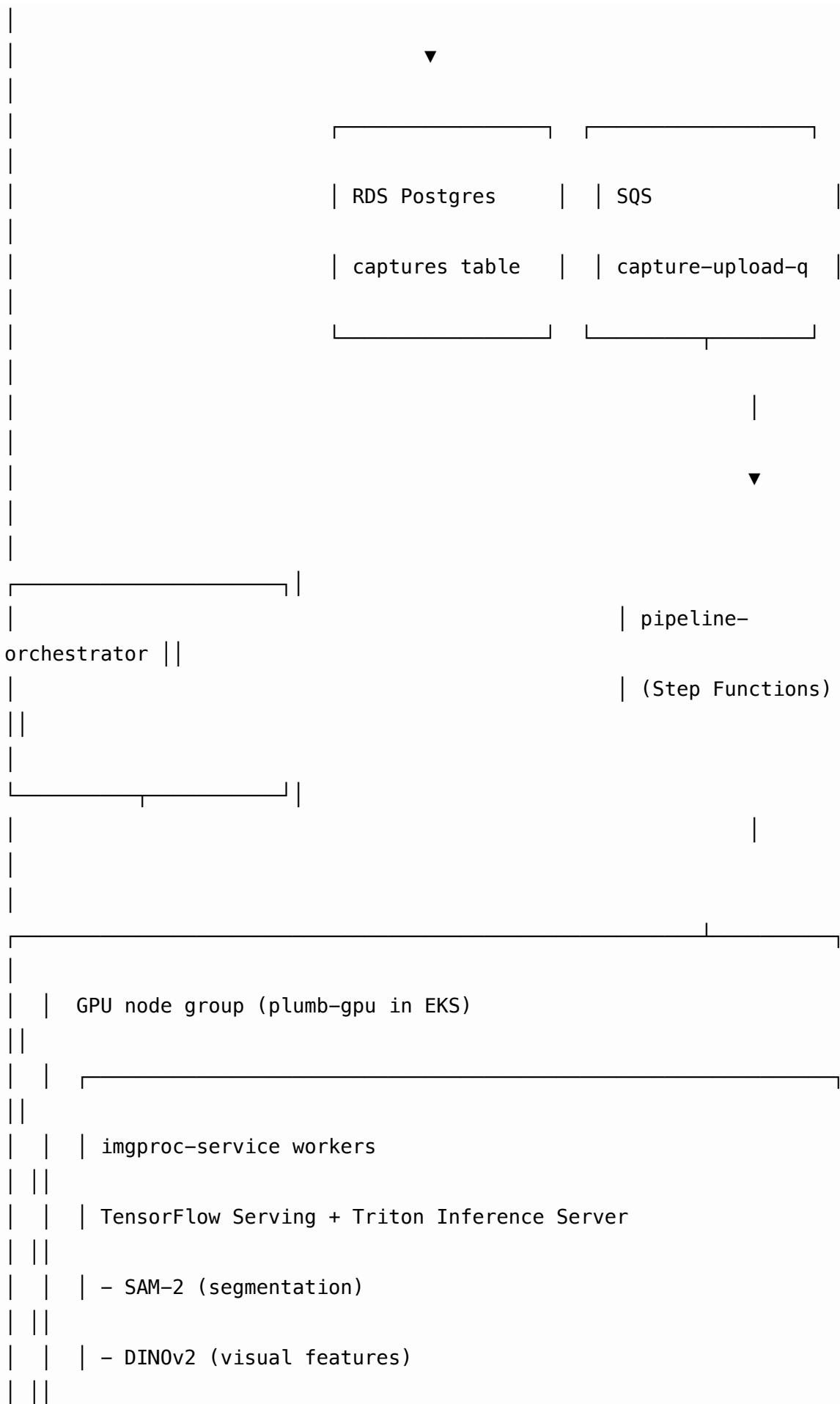
Key properties: - All CloudFront → ALB traffic is HTTPS (TLS 1.3) - API Gateway validates the Cognito JWT on every request - `x-request-id` is propagated through every hop (per `@plumb/observability`) - Next.js SSR fetch uses `force-dynamic` so the home page is always fresh - HTML is CDN-cached for 60s; API responses are not cached

5.2 Field user starts a 360° capture

A superintendent in the field, on iPhone with Bluetooth-paired Insta360 X4, taps “Start walk” in the mobile app.



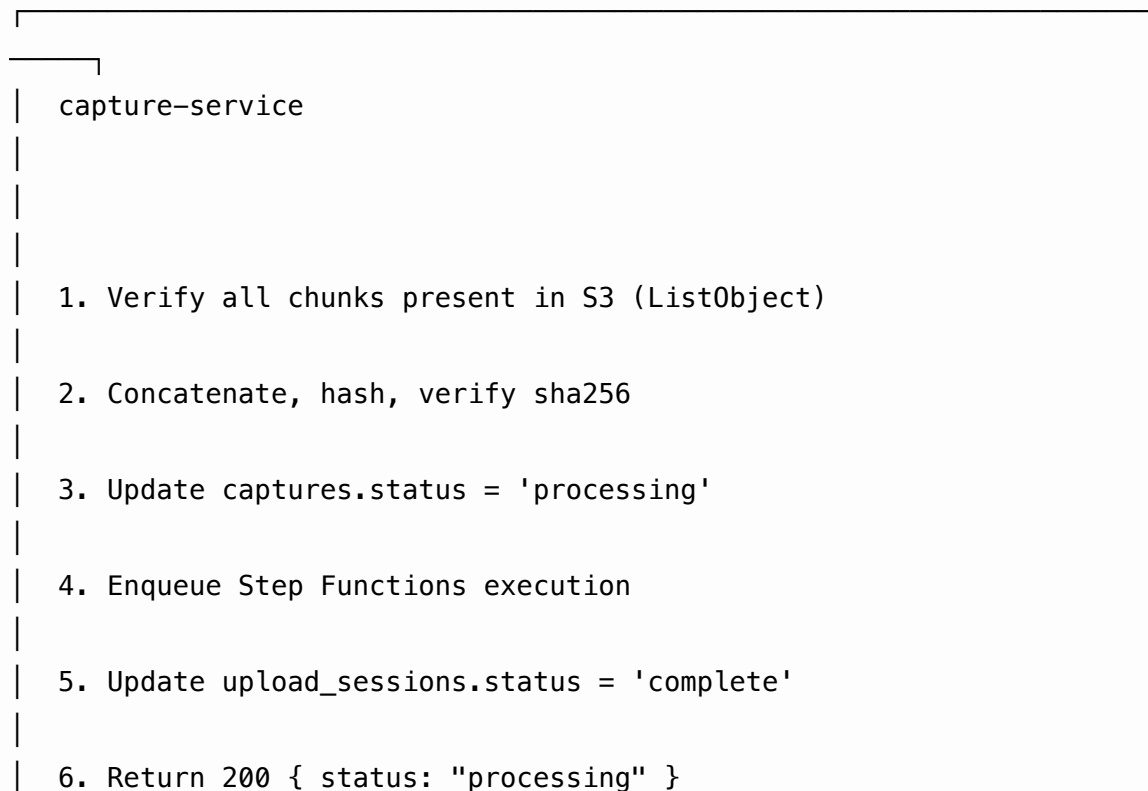
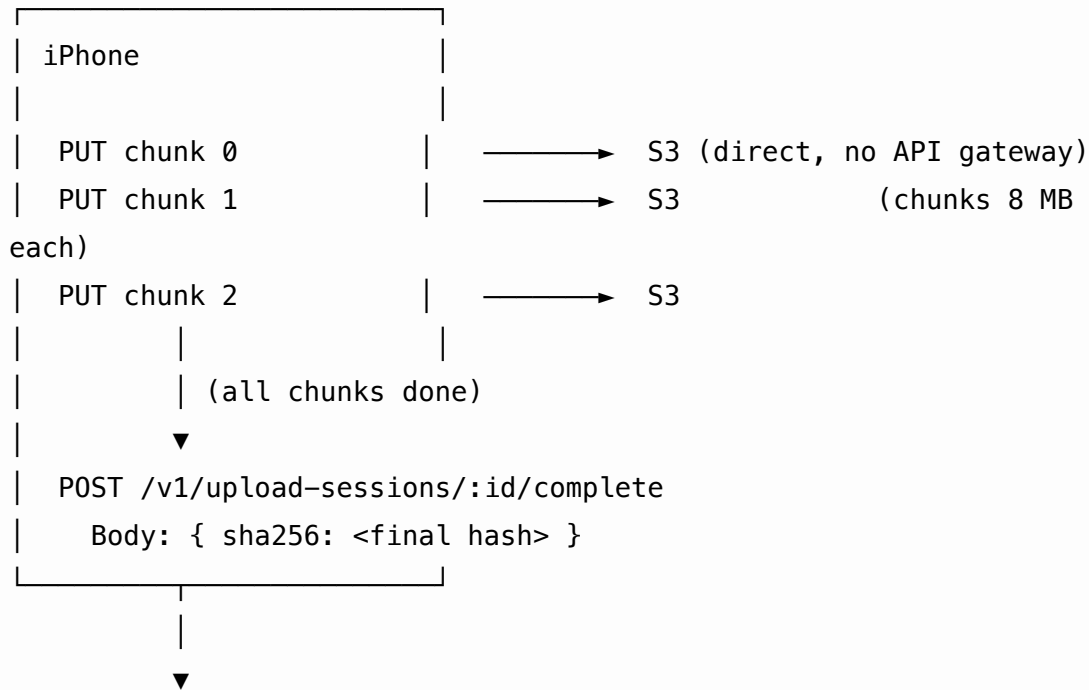




retry the entire initiation on flaky network without creating duplicate captures -
Pipeline is async: the mobile gets back immediately with an upload session;
spatial AI runs in the background on GPU nodes

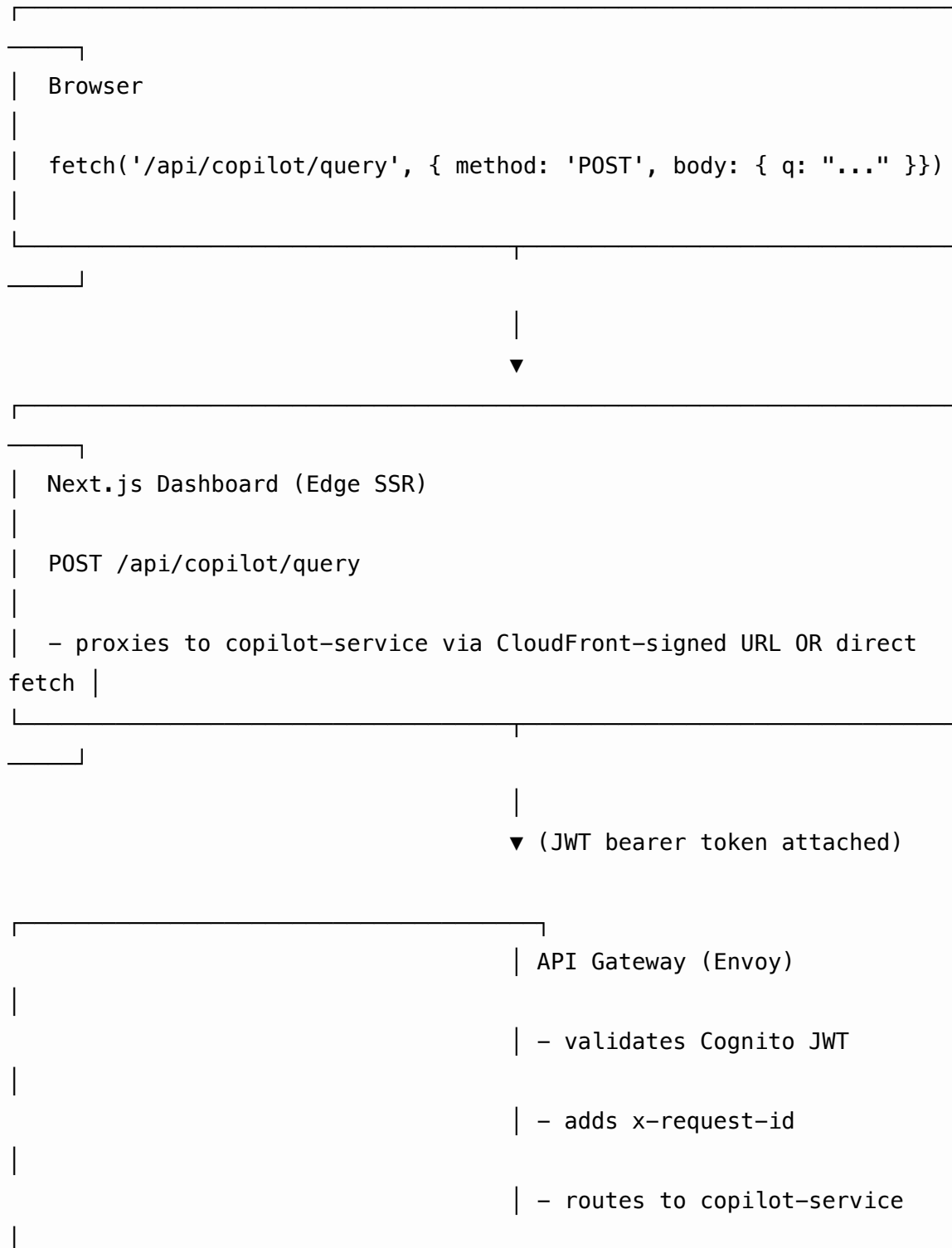
5.3 Capture upload — chunk-by-chunk

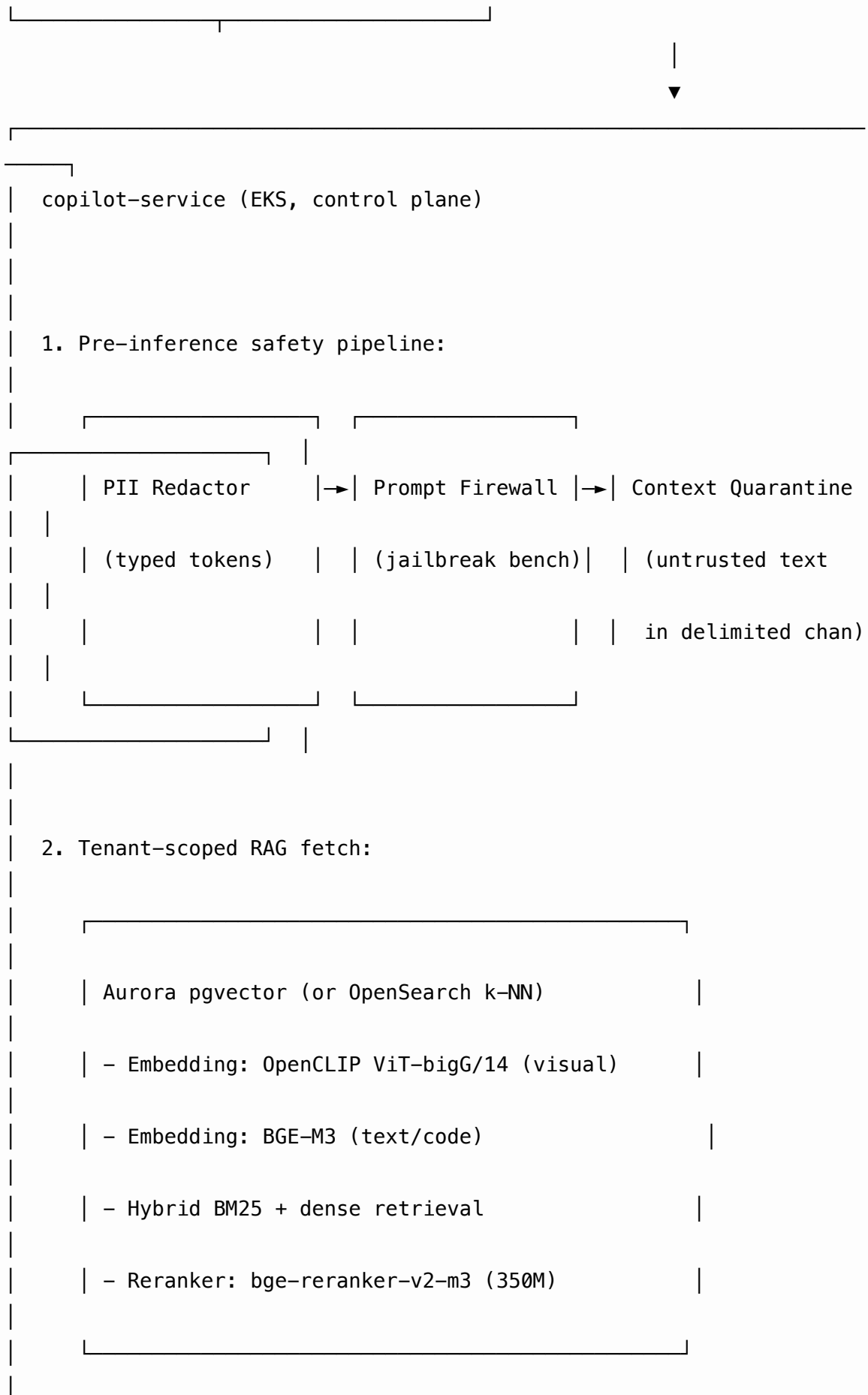
Continuing from §5.2, after the mobile gets the pre-signed URLs:



5.4 AI Copilot query

A user in the dashboard types “show me open RFIs on Level 3 over \$5k”.





3. LLM inference (function-calling):

Bedrock (Claude) OR Self-hosted Llama 3 70B

Routing:

- US-GovCloud → Claude on GovCloud
- EU → Mistral on SageMaker (EU region)
- JP/KSA → Sakura-hosted Llama

4. Streaming SSE chunks → browser:

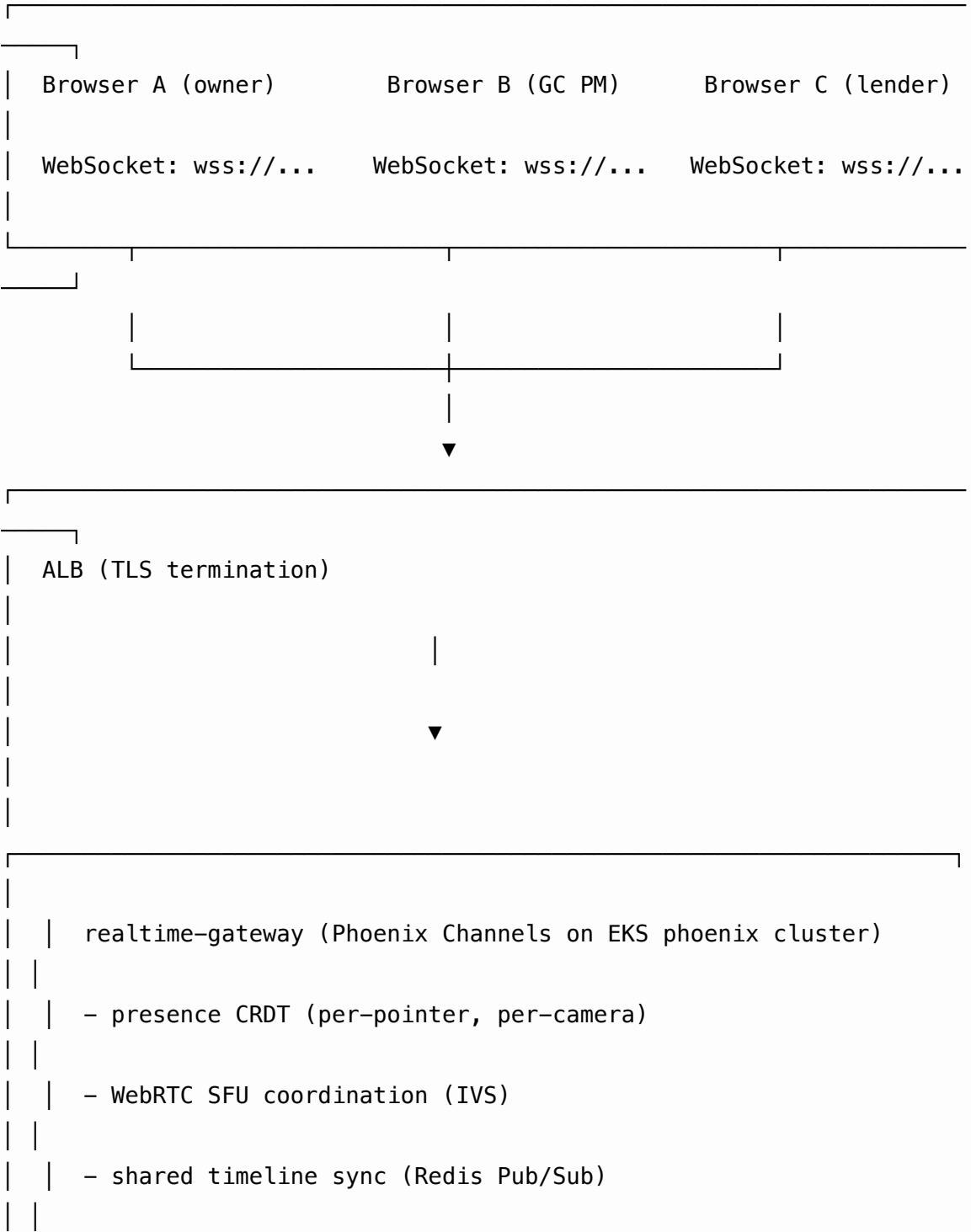
```
{ delta: "I found 3 RFIs on Level 3 over $5k:\n",
  citations: [
    { type: "rfi", id: "rfi_00000007", title: "Slab
thickness...",
      thumbnail_url: "...",
      geo: { lat: 40.7, lon: -74.0 } }
  ] }
```

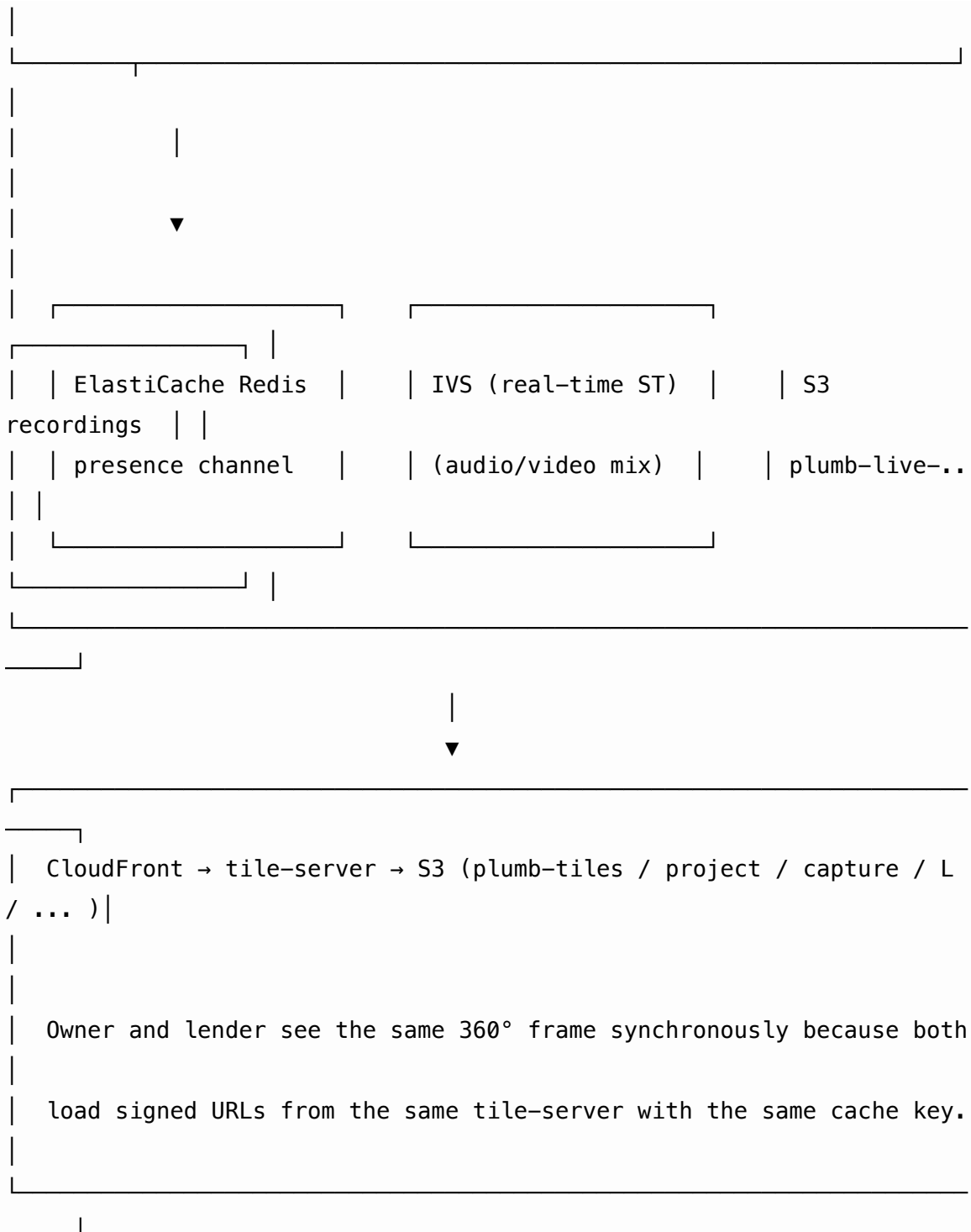
Key properties: - **Safety pipeline runs before AND after the model** (PII redactor on the way in, content classifier on the way out, C2PA provenance stamped on derived assets) - **Citations are mandatory** — refusal-on-no-citation policy; the model cannot generate a claim without a citation from the project

graph - **Streaming SSE** keeps the browser responsive even for multi-second retrieval-then-reason cycles - **Tenant-locked retrieval** — the vector store has per-tenant key prefixes enforced at the row level; cross-tenant leakage is impossible

5.5 Multi-stakeholder live walkthrough

The owner, the GC PM, and a lender are all in the same live 360° walkthrough.

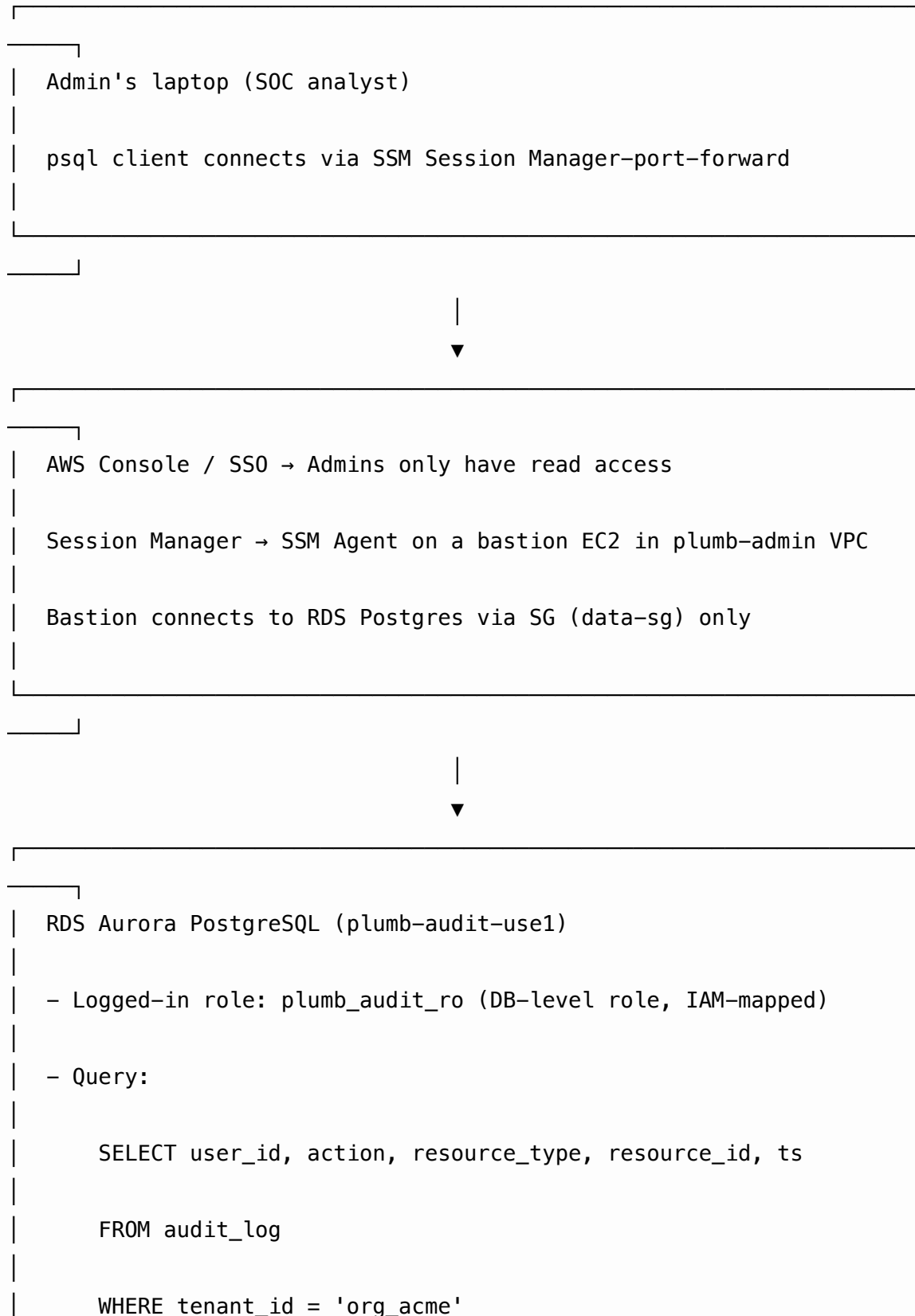




Key properties: - **Phoenix Channels** for soft-realtime presence (chosen for the BEAM VM's fault tolerance and natural fan-out) - **IVS (Interactive Video Service)** for real-time audio/video - **Redis Pub/Sub** as the presence bus (channel-per-walkthrough) - **S3** for recordings (plumb-live-{region}/...); auto-minutes via Transcribe → Bedrock summary - **Tile-server** is the same component used for non-live walkthroughs; the shared cache means owners and lenders see the same frame

5.6 Admin runs a SOC 2 audit query

The compliance team needs a query to answer: *“Show every access to a record owned by ACME Corp in the last 90 days”*.



```
AND ts > NOW() - INTERVAL '90 days'
```

```
ORDER BY ts DESC;
```

- Returns: 14,288 rows
- Each row exports to S3 + CloudTrail logs the access

```
AND ts > NOW() - INTERVAL '90 days'
```

```
ORDER BY ts DESC;
```

- Returns: 14,288 rows
- Each row exports to S3 + CloudTrail logs the access

```
AND ts > NOW() - INTERVAL '90 days'
```

```
ORDER BY ts DESC;
```

- Returns: 14,288 rows
- Each row exports to S3 + CloudTrail logs the access

```
AND ts > NOW() - INTERVAL '90 days'
```

```
ORDER BY ts DESC;
```

- Returns: 14,288 rows
- Each row exports to S3 + CloudTrail logs the access

```
|
|
| S3: plumb-audit-exports (Object Lock: Compliance mode, 7-year
retention)|
| - Query export parquet / CSV
|
| - CloudTrail logs the psql connection and query
|
| - KMS-encrypted at rest
|
```

```
|
|
| S3: plumb-audit-exports (Object Lock: Compliance mode, 7-year
retention)|
| - Query export parquet / CSV
|
| - CloudTrail logs the psql connection and query
|
| - KMS-encrypted at rest
|
```

```
|
|
| S3: plumb-audit-exports (Object Lock: Compliance mode, 7-year
retention)|
| - Query export parquet / CSV
|
| - CloudTrail logs the psql connection and query
|
| - KMS-encrypted at rest
|
```

```
|
|
| S3: plumb-audit-exports (Object Lock: Compliance mode, 7-year
retention)|
| - Query export parquet / CSV
|
| - CloudTrail logs the psql connection and query
|
| - KMS-encrypted at rest
|
```

Key properties:

- **No standing access.** Admins use SSM Session Manager, never long-lived credentials
- **CloudTrail** records every API call (including psql queries, which generate RDS API calls)
- **Object Lock + Compliance mode** prevents audit export tampering
- **DB-level role separation** (`plumb_audit_ro`) prevents admins from masquerading as application users

6. Data Architecture

6.1 Logical data model

SYSTEM OF RECORD

RDS Aurora PostgreSQL 16

orgs	projects	users	sessions
id	id	id	id
name	org_id (FK)	email	user_id (FK)
region	name	org_id (FK)	token_hash
plan	status	role	issued_at
created_at	address	display_name	expires_at
	created_at	created_at	

org_memberships	project_members	issues / RFIs
user_id (FK)	user_id (FK)	project_id (FK)
org_id (FK)	project_id (FK)	type
role	role	status
created_at	created_at	assignee_id (FK)
		created_at

captures	capture_frames	audit_log
id	id	id
project_id (FK)	capture_id (FK)	tenant_id
status	pose ENU	user_id
created_at	ts_ms	action
ingest_status	tile_path	resource_type
		resource_id
		trace_id
		ts
field_notes		signed_hash (HMAC)
project_id (FK)		
author_id (FK)		
body (text)		
body_audio (S3)	pipeline_runs	
created_at	id	
	capture_id (FK)	
	stage	
	status	
integration_	started_at	abac_policies

Bucket	Purpose	Lifecycle	Encryption
plumb-drone-{region}	Drone telemetry logs, orthomosaics	Hot 90d → IA 365d	SSE-KMS
plumb-live-{region}	Live walkthrough recordings	Hot 90d → IA 365d	SSE-KMS
plumb-audit-exports-{region}	Audit query exports (Object Lock Compliance, 7y)	Never (locked)	SSE-KMS + Object Lock
plumb-log-archive-{region}	Long-term log archive (S3 Standard-IA after 30d)	Hot 30d → IA 365d	SSE-KMS
plumb-customer-exports-{region}	Per-tenant export buckets (BYO data)	Customer-defined	Customer CMK

All buckets are private. Access via CloudFront signed URLs or S3 access points only. Cross-region replication active for `plumb-raw-360-` and `plumb-audit-exports-` (encryption-at-rest is preserved across regions).

6.3 TimescaleDB (capture telemetry)

TimescaleDB (ext. on RDS Aurora)	
hypertable: capture_telemetry	
ts	timestampz (partition key)
capture_id	text
user_id	text
event_type	text (gps, imu, ar_drop, ...)
payload	jsonb
hypertable: device_pings	
ts	timestampz
user_id	text
battery_pct	smallint

thermal_state	text
---------------	------

hypertable: sync_events	
ts	timestampz
device_id	text
user_id	text
event	text
bytes_synced	bigint

Retention: 90 days hot, 2 years compressed (continuous aggregates).

6.4 ClickHouse (analytics)

ClickHouse Cloud (or Redshift Serverless)	
events	
ts	DateTime
tenant_id	String
user_id	String
event_type	LowCardinality(String)
properties	String (JSON)

rum (real user metrics – frontend telemetry)	
ts	DateTime
session_id	String
route	String
lcp_ms	UInt32
fid_ms	UInt32
cls	Float32

cost_rollup	
tenant_id	String
service	String
day	Date
ai_minutes	Float64
egress_gb	Float64
storage_gb	Float64

6.5 Redis usage

Cluster	Engine	Purpose	Eviction	TTL
plumb-session-{region}	Redis 7	Session store, idempotency cache, idempotency keys	allkeys-lru	1h sessions, 24h idem
plumb-presence-{region}	Redis 7	Phoenix Channels presence pub/sub	volatile-lru	ephemeral
plumb-rate-{region}	Redis 7	Token-bucket rate limiting per (userId, route)	volatile-lru	1m
plumb-flow-{region}	Redis 7	Idempotency and dedup keys for upstream integrations	volatile-lru	24h

Cluster mode: 6 shards × 2 replicas per primary. AOF persistence on plumb-session- and plumb-flow- only.

7. Compute Architecture

7.1 EKS cluster topology

Cluster	Roles	Node groups	Region deployment
plumb-control-use1	All stateless services (Phase 0 + Phase 1 control)	system, app, gpu	us-east-1

Cluster	Roles	Node groups	Region deployment
plumb-realtime-use1	Phoenix Channels realtime gateway	realtime	us-east-1
plumb-gpu-use1	imgproc-service workers (P5/G5)	gpu	us-east-1
plumb-batch-use1	Step Functions, ETL, scheduled jobs	batch	us-east-1

Mirrored in eu-west-1, ap-southeast-2, ap-northeast-1, ksa-central-1.

7.2 Node groups

Group	AMI	Capacity	Disk	Use
system	Bottlerocket EKS-optimized	t4g.medium × 3	20 GB gp3	Karpenter, CoreDNS, Ingress-NGINX
app	Bottlerocket EKS-optimized	m6i.xlarge × 3 (min), 30 (max)	100 GB gp3	Stateless services
app-arm	Bottlerocket (arm64)	m6g.xlarge × 3 (min), 20 (max)	100 GB gp3	Cost-optimized for low-CPU services
gpu-a10g	DLAMI	g5.xlarge × 1 (min), 8 (max)	500 GB gp3	SAM-2 inference (online)
gpu-a100	DLAMI	p4d.24xlarge × 0 (min), 4 (max)	1 TB gp3	3DGS training (batch)
gpu-h100	DLAMI	p5.48xlarge × 0 (min), 2 (max)	2 TB gp3	Heavy batch (LLM fine-tuning, model upgrades)

Group	AMI	Capacity	Disk	Use
realtime	Bottlerocket	c6i.2xlarge x		Phoenix
	EKS-optimized	3 (min), 12 (max)	50 GB gp3	Channels cluster
batch	Bottlerocket	m6i.2xlarge x		Step
	EKS-optimized	0 (min), 10 (max)	100 GB gp3	Functions, ETL

Karpenter manages the app, app-arm, gpu-a10g, gpu-a100, gpu-h100, and batch node groups. **Cluster Autoscaler** manages system and realtime (predictable baseline).

7.3 Deploy topology per service

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: org-service
  namespace: plumb
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    spec:
      serviceAccountName: org-service
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
      containers:
        - name: org-service
          image: <account>.dkr.ecr.us-east-1.amazonaws.com/org-service:v0.1.0
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }

```

```

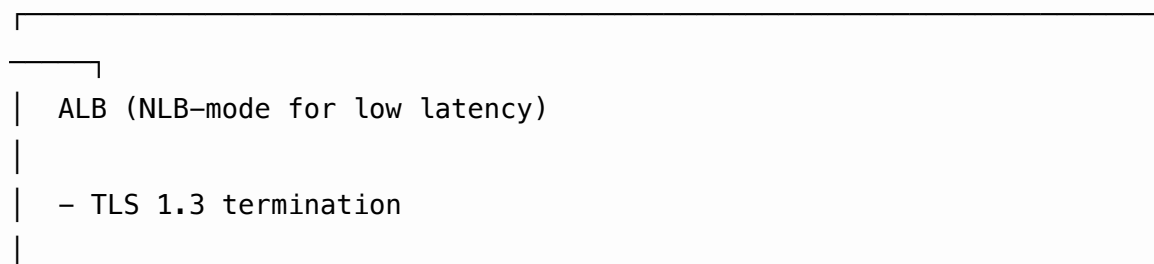
readinessProbe:
  httpGet: { path: /v1/health, port: 8080 }
  initialDelaySeconds: 5
  periodSeconds: 5
livenessProbe:
  httpGet: { path: /v1/health, port: 8080 }
  initialDelaySeconds: 30
  periodSeconds: 30
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: org-service-secrets
        key: database-url
  - name: SERVICE_NAME
    value: org-service
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: [ALL]

```

Key properties: - **Zero-trust pod:** non-root, read-only root FS, all capabilities dropped - **IRSA (IAM Roles for Service Accounts):** the `org-service` ServiceAccount is annotated to assume an IAM role via OIDC, so the pod can use `aws-sdk` without static credentials - **Topology spread:** even distribution across AZs - **Rolling updates** with `maxUnavailable: 0` = zero-downtime deploys

7.4 Ingress

API Gateway → ALB (TLS 1.3) → Ingress-NGINX → Service.



- ACM cert for *.plumb.dev
- WAF integration (managed rule sets)
- Target group: EKS Ingress-NGINX



- Ingress-NGINX (DaemonSet on EKS system nodes)
- WAF header pass-through
 - Mutating x-request-id (if missing) → propagates from API Gateway
 - Routes:
 - /v1/* → plumb-control namespace (stateless services)
 - /ws → plumb-realtime namespace (Phoenix Channels)
 - /stream → plumb-gpu namespace (imgproc-service)
 - /admin/* → plumb-admin namespace (RBAC enforced)
 - /api/* → plumb-dashboard namespace (dashboard SSR)
 - Rate limiting via NGINX annotations (defense-in-depth)

7.5 Step Functions (capture pipeline)

Step Functions state machine: CapturePipeline

StartAt: DecodeFrame

DecodeFrame → ExtractFrames (Lambda)

ExtractFrames → SuperPointFeatures (Lambda + GPU)

SuperPointFeatures → SfM_GLOMAP (ECS RunTask)

SfM_GLOMAP → MVS_OpenMVS (ECS RunTask)

MVS_OpenMVS → MeshPoisson (ECS RunTask)

MeshPoisson → ThreeDGS (ECS RunTask)

ThreeDGS → SegmentSAM2 (ECS RunTask)

SegmentSAM2 → AlignToBIM (ECS RunTask)

AlignToBIM → Publish (Lambda)

Publish → SNS → capture-service (update DB + emit WebSocket)

Failure handling:

- Any state error → Catch → DLQ (SQS) → on-call page
- SfM/MVS retries with smaller image pool after 3 failures
- User visible state: 'processing' → 'succeeded' or 'failed'

8. Networking & Service Mesh

8.1 Service mesh: App Mesh vs Istio (decision)

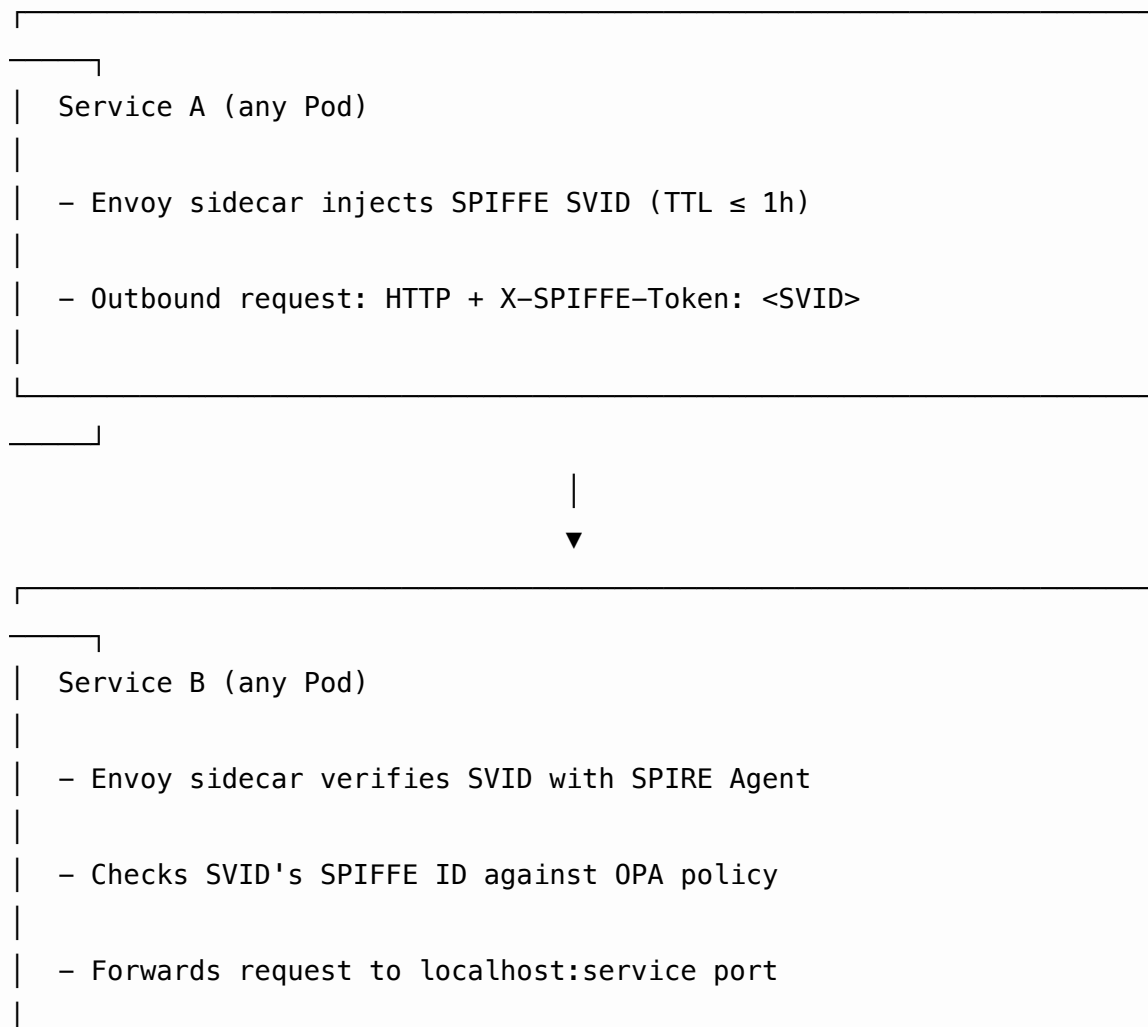
Decision: App Mesh with Envoy sidecars, *not* Istio.

Reasons: - App Mesh is **AWS-native** (no third-party control plane to operate) - **Lower operational cost** (no Istio control plane, no extra RBAC) - **Native integration** with X-Ray, CloudWatch, IAM - **AWS-supported** — backed by AWS engineering - Master plan §10 specifies “service mesh only where operationally justified (not added by default)”

For Phase 0 and Phase 1, we don’t deploy a mesh. East-west traffic is HTTP over the cluster-internal network. App Mesh becomes relevant at Phase 2 when mTLS + SPIFFE is mandated.

8.2 Service-to-service auth (Phase 2+)

When mTLS is mandated:



Deployment: SPIRE Server runs as a StatefulSet; SPIRE Agent runs as a DaemonSet; each service has a SPIRE Registration Entry. The @plumb/auth package already implements the verify side (§4.6 of PLUMB-PHASE-0-REPORT.md); the SPIRE integration is a Phase 2 wiring task.

8.3 Inter-AZ traffic

- All services run in **3 AZs** for HA
- Aurora PostgreSQL uses **Multi-AZ** with sync replicas
- ElastiCache Redis uses **Multi-AZ** with automatic failover
- S3 is **regional** (multi-AZ transparent)
- OpenSearch uses **Multi-AZ** with 3 dedicated master nodes

8.4 Inter-region traffic

- **S3 Cross-Region Replication** for plumb-raw-360- and plumb-audit-exports-
- **Aurora Global Database** for cross-region read replicas (RPO ≤ 1s)
- **Route 53 latency-based routing** across regions
- **Transit Gateway peering** between commercial regions (us-east-1 ↔ us-west-2 ↔ eu-west-1 ↔ ap-southeast-2 ↔ ap-northeast-1)
- **GovCloud (us-gov-east-1)** is isolated — no transit peering to commercial

9. Identity & Access

9.1 User identity (Cognito + external IdP)

Customer Identity Provider
(Okta / Entra ID / Auth0 / Ping)

| OIDC / SAML SSO



- Amazon Cognito User Pool (per region)
- Federated Identity Providers (one per tenant)
- Per-tenant attribute mapping (org_id, role)
- MFA: Cognito MFA + WebAuthn (FIDO2) for admins



- JWT issued to mobile/web client
- Issuer: `https://cognito-idp.{region}.amazonaws.com/{userPoolId}`
- Audience: `plumb-api`
- Lifetime: 1 hour (access token), 30 days (refresh token)
- Claims: `sub`, `email`, `org_id`, `role`, `custom:tenant_id`

Verification flow: 1. User logs in via their IdP (Okta / Entra) 2. IdP returns code to mobile/web 3. Mobile/web exchanges code for Cognito JWT 4. Mobile/web sends JWT as `Authorization: Bearer <jwt>` to API Gateway 5. API Gateway validates JWT signature, claims, expiry 6. Cognito user pool lookup populates `req.principal`

9.2 Workload identity (IAM Roles for Service Accounts)

Every EKS ServiceAccount is annotated with an IAM role. This is the **AWS-native workload identity** mechanism; SPIFFE is the **cross-cloud** alternative.

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: org-service
  namespace: plumb
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/plumb-
      org-service

```

The IAM role has a trust policy that only allows the org-service SA to assume it (via OIDC issuer `oidc.eks.us-east-1.amazonaws.com/...`). It has permissions scoped to exactly what the service needs:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:org-service-*"
    },
    {
      "Effect": "Allow",
      "Action": ["rds-db:connect"],
      "Resource": "arn:aws:rds:us-east-1:123456789012:db:plumb-control"
    }
  ]
}

```

9.3 Tenant isolation

Tenant isolation is enforced at **five layers**:

1. **Database:** every row has `tenant_id`; row-level security policies filter rows by `current_setting('app.tenant_id')`
2. **Redis:** keys are prefixed with `tenant_id`; ACLs disallow cross-tenant access
3. **S3:** objects have `tenant_id` in the key; access points have per-tenant policies
4. **KMS:** every tenant has a Customer Managed Key (CMK); per-object encryption uses the tenant's CMK

5. **API Gateway:** JWT has tenant_id claim; every service routes it through to DB queries

10. Observability

10.1 The three pillars

METRICS

- CloudWatch Metrics (AWS-native)
- Prometheus + Grafana (SLO dashboards)
- 1s resolution for control-plane alerts
- Custom metrics: SLO burn rate, error budget, queue depth

LOGS

- Structured JSON via @plumb/observability
- CloudWatch Logs (control plane)
- Kinesis Firehose → S3 (long-term archive)
- OpenSearch (full-text search, ad-hoc)
- Every log line carries request_id, service, ts, level, msg

TRACES

- OpenTelemetry SDK in every service
- AWS X-Ray collector (DaemonSet)
- 2.5% sampling default, 100% sampling for errors
- Trace context propagated via W3C `traceparent` header

10.2 SLO definitions

Service	SLI	SLO	Error budget
All HTTP APIs	p99 latency	< 500ms	0.1% per month
All HTTP APIs	availability	99.9%	43m/month
Capture ingest	chunk upload success	99.5%	0.5% per month
Spatial AI pipeline	capture-to-publish latency	p50 < 1.3h, p95 < 3h	n/a (latency-bound)
Copilot	query latency (p95)	< 4s	n/a
Realtime gateway	WebSocket connection stability	99.5%	0.5% per month

10.3 Alert routing

SEV1 (production down, data loss, security breach) → PagerDuty → on-call SRE

→ 5 min ack, 30 min mitigate

SEV2 (major feature broken, SLO burn > 2x) → PagerDuty → on-call

engineer

→ 15 min ack, 4h mitigate

SEV3 (minor broken, SLO burn > 1x) → Slack #oncall, no page

→ next business day

SEV4 (cosmetic, observation) → Linear ticket, no notification

10.4 Cost observability

Per-tenant cost attribution via Cost and Usage Reports (CUR) + Athena:

-- per-tenant, per-service, per-day cost

SELECT

tenant_id,

line_item_product_code,

DATE(line_item_usage_start_date) AS day,

SUM(line_item_unblended_cost) AS cost_usd

FROM plumb_cur

WHERE line_item_usage_start_date >= NOW() - INTERVAL '30 days'

GROUP BY 1, 2, 3

ORDER BY day DESC;

Per-tenant dashboard (Grafana) shows: - AI-minutes used (from service-level metrics) - Storage GB (from S3 inventory) - Egress GB (from CloudFront logs) - Total cost (from CUR)

11. Security & Compliance

11.1 Layered defenses

Layer	Defense	AWS Service
Edge	WAF managed rule sets, rate limiting, geo-restriction	AWS WAF, Shield Advanced
Transport	TLS 1.3 everywhere, ACM cert rotation	ACM, CloudFront
Authentication	Cognito + OIDC + SAML SSO + MFA	Cognito

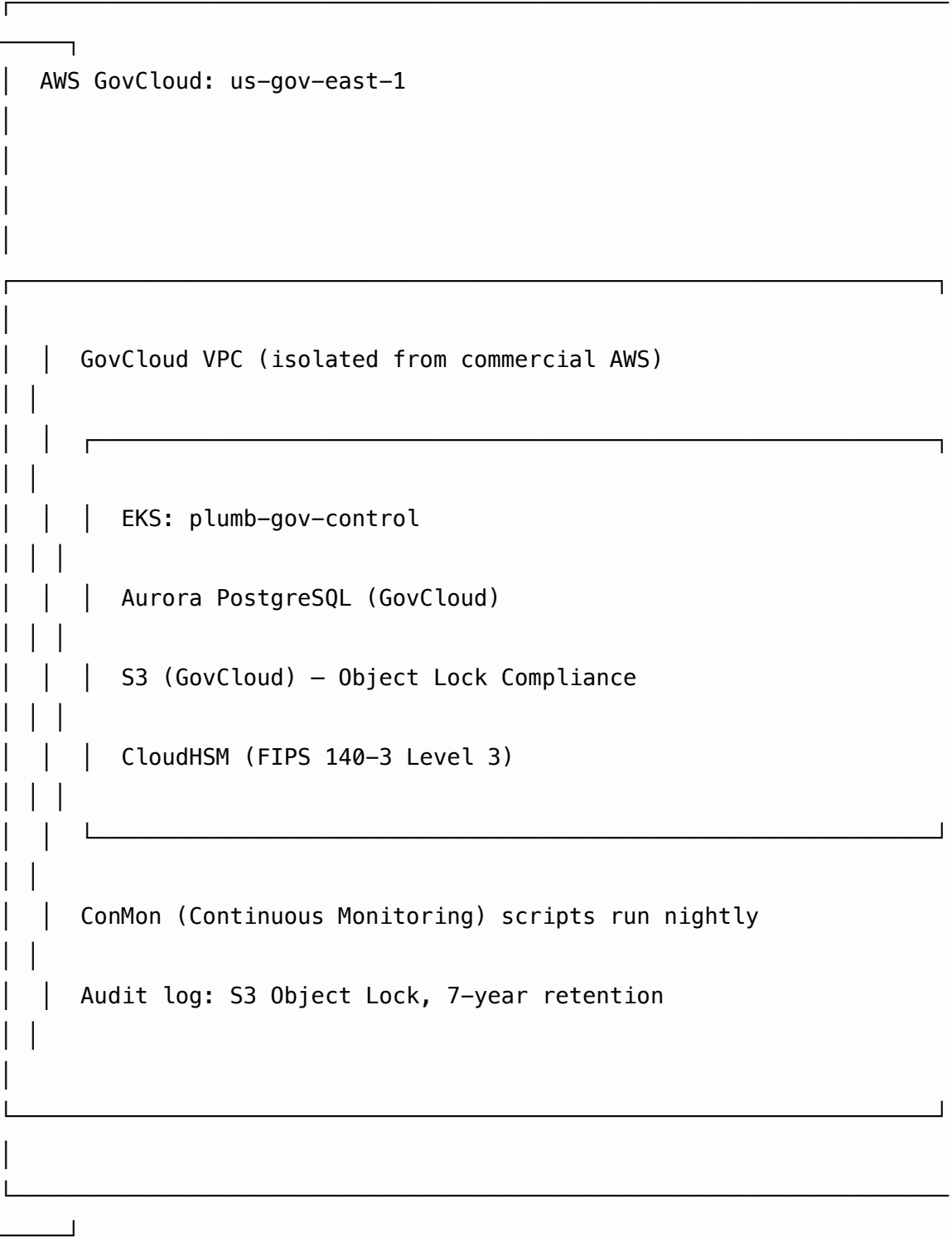
Layer	Defense	AWS Service
Authorization	RBAC + ABAC + OPA policies	OPA (in EKS)
Network	VPC isolation, security groups, VPC endpoints	VPC, PrivateLink
Workload	SPIFFE/SVID mTLS (Phase 2+)	SPIRE
Data at rest	SSE-KMS with per-tenant CMK	KMS, CloudHSM (root)
Data in transit	TLS 1.3, mTLS (Phase 2+)	ACM, SPIRE
Secrets	AWS Secrets Manager, auto-rotation 90d	Secrets Manager
Application	SAST, DAST, SCA, dep scanning, SBOM	GitHub Actions, SonarQube
Audit	CloudTrail, S3 Object Lock Compliance	CloudTrail, S3
Penetration testing	Quarterly third-party	(external vendor)

11.2 Compliance posture

Compliance	Status	Target	AWS services used
SOC 2 Type II	In progress (Phase 1)	Type II report Q2 2027	CloudTrail, Config, IAM
ISO 27001	In progress (Phase 2)	Q4 2027	(same as SOC 2)
FedRAMP Moderate	In progress (Phase 2)	Authorization Q4 2027	GovCloud, CloudHSM, ConMon
HIPAA	In progress (Phase 3)	Q2 2028	(same as SOC 2)
GDPR	Compliance-ready	Continuous	EU region, data residency, DSR workflows

Compliance	Status	Target	AWS services used
CCPA/CPRA	Compliance-ready	Continuous	(same as GDPR)

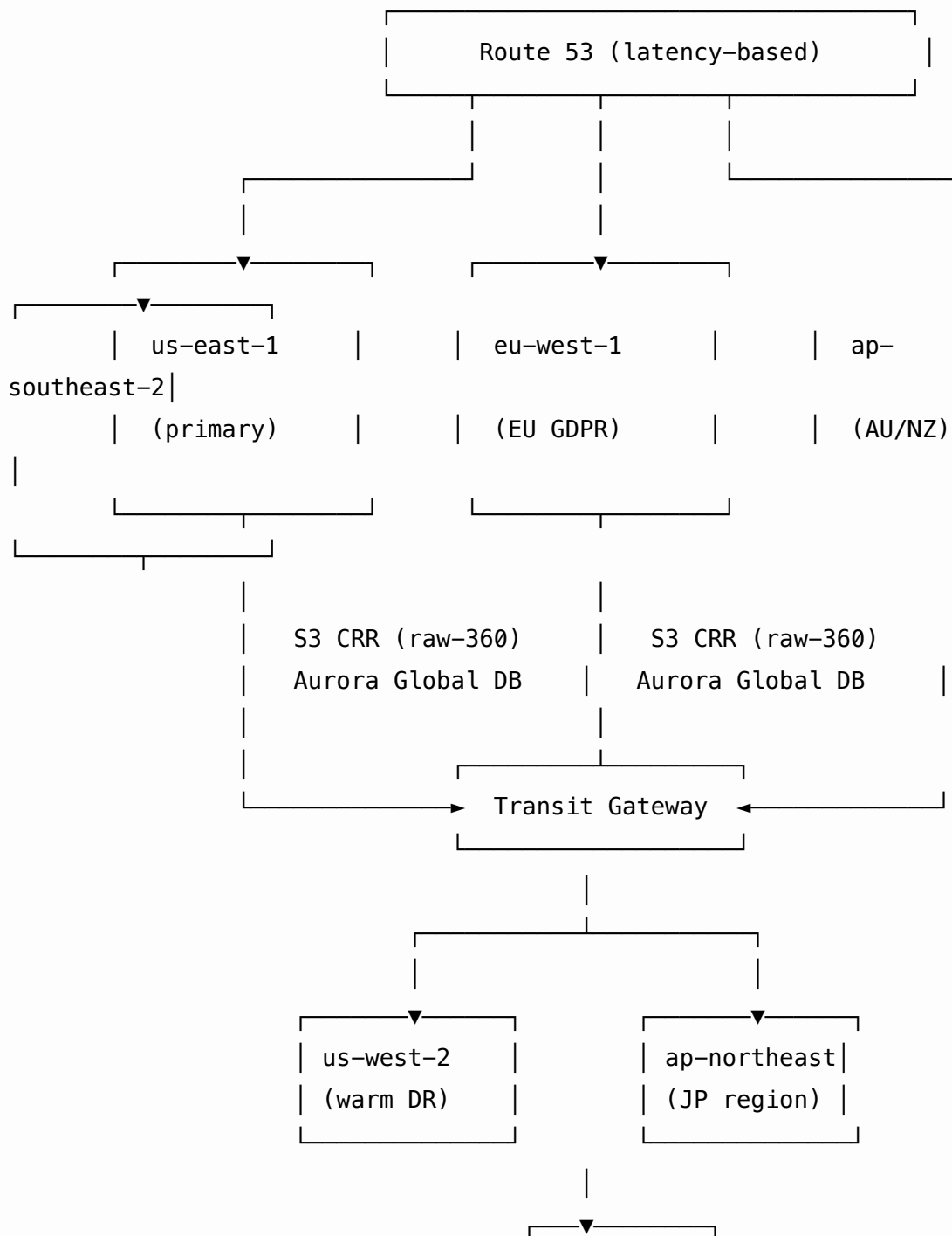
11.3 FedRAMP boundary (us-gov-east-1)



FedRAMP authorization requires: - 12–18 months of evidence collection -
ConMon team hired before GA - Annual penetration test by FedRAMP-accredited
3PAO - POA&M (Plan of Action & Milestones) tracking

12. Multi-Region & Disaster Recovery

12.1 Region topology



12.2 RPO / RTO targets

Service tier	RPO	RTO	Strategy
Tier 1 (control plane APIs)	15 min	1 hour	Multi-AZ + Aurora Global DB
Tier 2 (capture pipeline)	1 hour	4 hours	S3 CRR + Step Functions replay
Tier 3 (analytics, audit logs)	4 hours	24 hours	S3 IA + Glacier + lazy replication

12.3 Active-active vs active-passive

Layer	Strategy
API Gateway	Active-active (Route 53 latency-based)
Aurora Postgres	Active-active for reads (Global DB), active-passive for writes (single primary per region)
S3	Active-active (CRR)
Redis	Active-passive per region (Cluster Mode with replicas)
GPU pipeline	Active-passive (only one region runs the pipeline at a time, others stand by)

12.4 DR drill cadence

- Quarterly: full-region failover drill (simulated by Route 53 health check failure or maintenance page)
- Monthly: Aurora Global DB promote read replica to primary
- Weekly: S3 cross-region replication lag check
- Daily: automated backup integrity check

13. Cost Model

Per-tenant unit economics on AWS, scaled to 1,000 active customers.

13.1 Variable cost per capture

Component	Service	Cost per capture
Storage (raw 360, 5 GB avg)	S3 Standard → IA after 30d	$\$0.012/\text{GB} \times 5 \text{ GB} = \0.060
Compute (decode + SfM + MVS + 3DGS)	Lambda + EC2 GPU spot	$\$0.65 / \text{capture}$ (12 min p5.4xlarge avg)
Embeddings (BGE-M3 + OpenCLIP)	Bedrock Titan	$\$0.002 / \text{capture}$
Inference (Copilot when used)	Bedrock Claude	$\$0.14 / \text{active user} / \text{day}$
CDN egress	CloudFront	$\$0.05 / \text{GB} \times 2 \text{ GB avg} = \0.10
Step Functions	Step Functions	$\$0.025 \text{ per state transition}$
Total per capture		~\$0.95

13.2 Per-tenant monthly cost (Pro tier)

Component	Service	Cost per tenant/month
Seat (5 users avg)	Internal allocation	$\$25 / \text{user} / \text{month} \times 5 = \125
100 captures / month	Per-capture cost	$\$0.95 \times 100 = \95
250 GB raw storage	S3 Standard	\$3
1 TB derived storage	S3 IA	\$12.50
AI Copilot (moderate use)	Bedrock Claude	\$15
Lambda invocations	10M / month	\$2

Component	Service	Cost per tenant/month
RDS compute (allocation)	Aurora	\$50
Redis (allocation)	ElastiCache	\$10
EKS compute (allocation)	EKS	\$30
CloudWatch + X-Ray	Observability	\$5
Data transfer	CloudFront	\$8
Subtotal		~\$355
Reserved Instance / Savings Plan discount (30%)		-\$107
Net variable cost		~\$248
Retail price (\$480/seat × 5)		\$2,400
Gross margin		90%

13.3 Reserved capacity to plan for

Resource	Monthly reservation	Cost saving
RDS Aurora (db.r6g.4xlarge reserved × 3 AZs)	\$1,800 / month	35% vs on-demand
ElastiCache (cache.r6g.large reserved × 6 shards)	\$580 / month	30% vs on-demand
EKS compute (Savings Plan, 1-year)	\$4,500 / month	30% vs on-demand
EC2 GPU (P5 Capacity Block)	\$18,000 / month	25% vs on-demand

14. Deployment Topology

14.1 CI/CD pipeline

GitHub Actions (Phase 0 CI)

1. Lint (ESLint + Prettier)
 2. Typecheck (tsc)
 3. Unit tests (Vitest + node:test)
 4. Integration tests (Postgres service container)
 5. Boot smoke test (curl against running service)
 6. Build (tsc + Next.js)
 7. Push image to ECR
 8. Tag image with git SHA
 9. Generate SBOM (syft)
 10. Sign image (cosign)
-



ArgoCD (GitOps)

- Watches Git repo `infra/k8s/` for ArgoCD Applications
 - Auto-syncs to EKS clusters (prune + self-heal)
 - Manual approval gate for `prod` namespace
-

- Argo Rollouts for canary deployments:
 - setWeight: 1 → pause 5m → 5 → pause 10m → 25 → pause 15m → 50
 - pause 20m → 100
 - AnalysisTemplate: SLO burn rate, error rate, p95 latency
 - Auto-rollback on SLO burn > 2x
-

14.2 Argo Rollouts example (Phase 0 CI → Production)

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: org-service
  namespace: plumb
spec:
  replicas: 6
  strategy:
    canary:
      canaryService: org-service-canary
      stableService: org-service-stable
      trafficRouting:
        nginx:
          stableIngress: org-service
  steps:
    - setWeight: 1
    - pause: { duration: 5m }
    - setWeight: 5
    - pause: { duration: 10m }
    - analysis:
        templates:
          - templateName: org-service-slo
    - setWeight: 25
```

```

- pause: { duration: 15m }
- setWeight: 50
- pause: { duration: 20m }
- setWeight: 100
rollbackWindow:
revisions: 3

```

14.3 IaC layout

```

infra/
├─ terraform/
│   └─ modules/                # Reusable modules: VPC, EKS, RDS,
ElastiCache, S3
│   └─ us-east-1/              # Region-specific stacks
│   └─ eu-west-1/
│   └─ ap-southeast-2/
│   └─ ap-northeast-1/
│   └─ ksa-central-1/
│   └─ us-gov-east-1/          # FedRAMP boundary (isolated)
└─ k8s/
    └─ apps/                   # ArgoCD Application manifests
    └─ infra/                  # Karpenter, Ingress-NGINX, cert-manager
    └─ overlays/               # Customize overlays per environment

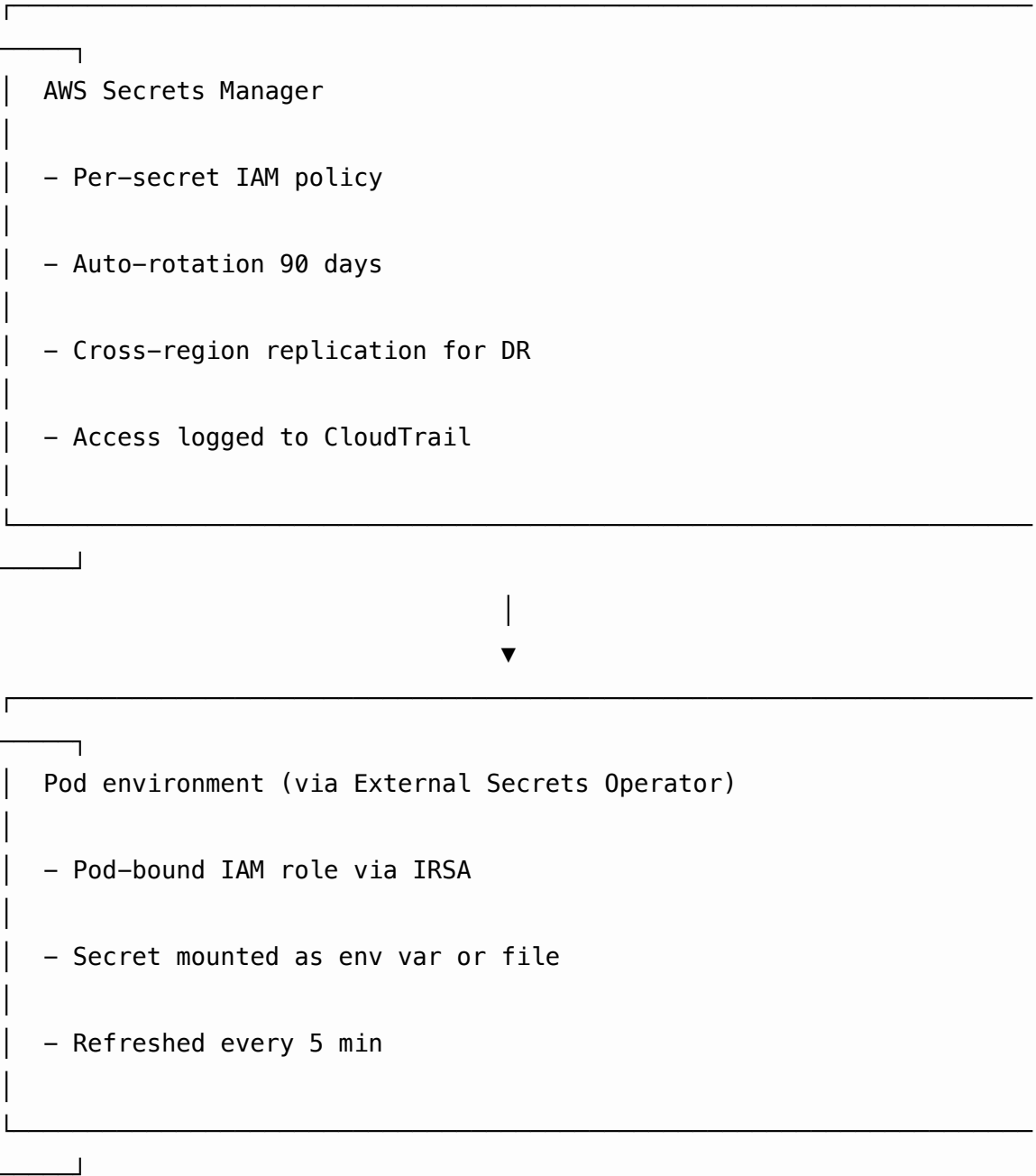
```

14.4 Promotion flow

PR opened against main

- GitHub Actions CI runs (lint, test, typecheck, build)
- On green: image pushed to ECR with tag "pr-{n}"
- ArgoCD sync to dev (manual promote, sandbox)
- Smoke test against dev
- Manual approve → Argo Rollouts canary to staging
- Integration tests against staging
- Manual approve → Argo Rollouts canary to prod
- SLO monitor watches for 1 hour
- Auto-rollback if SLO burn > 2x

14.5 Secrets management



15. Risks & Trade-offs

#	Risk	Severity	Mitigation
1	EKS cluster autoscaling under GPU load spikes	High	Karpenter MIG + multi-class GPU pool (A10G + A100 + H100) +

#	Risk	Severity	Mitigation
2	(large customer capture burst) Aurora RDS scaling bottleneck (project-membership fanout on a large org)	High	admission quotas per tenant Partition by tenant_id; read replicas; pgsat pooler; hot connection pool per query
3	S3 egress cost (large customers streaming 360° tiles)	High	CloudFront Origin Shield + per-tenant signed URLs + cache-control heads + tile pyramid LOD
4	Step Functions execution cost on retry storms	Medium	DLQ-on-failure + circuit breaker; not every state retries
5	CloudWatch log volume (high-cardinality metadata)	Medium	Sampling + structured logs only; routes to S3 + OpenSearch for ad-hoc
6	Multi-region data-residency complexity	High	Per-region tenant pinning; explicit residency policy engine; fail-closed routing
7	Bedrock rate limits during Copilot burst	Medium	Per-tenant token-bucket + LLM provider fallback (Llama self-hosted)
8	Phoenix Channels WebSocket	Medium	Cluster of phoenix nodes behind ALB;

#	Risk	Severity	Mitigation
9	scaling for live walkthroughs	Low	sticky sessions by walkthrough-id
	SPIFFE/SPIRE operational complexity (Phase 2)		Defer to Phase 2; track as Phase 2 deliverable
10	Cost attribution accuracy (CUR lag = 24h)	Low	Internal metering backed by service-level metrics for real-time cost

16. Open Decisions

These are decisions that need product / engineering / leadership input before this architecture becomes the implementation plan.

#	Decision	Options	Default	Owner
1	GPU mix for imgproc	A10G-only (cheaper) vs A100-mix (faster)	A10G-primary, A100-burst	Eng Lead
2	Vector store	pgvector (Phase 1) vs OpenSearch k-NN (Phase 2)	pgvector first, migrate if usage > 10M vectors	ML Lead
3	Mobile shell	Native (Swift + Kotlin) vs KMM	Native (matches master plan)	Mobile Lead
4	Realtime gateway tech	Elixir Phoenix vs Node + Socket.IO	Elixir Phoenix (matches master plan)	Eng Lead
5	Multi-region failover	Active-active per region vs	Active-active for Tier 1,	Eng Lead

#	Decision	Options	Default	Owner
	strategy	Active-passive with hot standby	active-passive for GPU	
6	Kubernetes distribution	EKS (chosen) vs self-managed K8s	EKS	Eng Lead
7	Container runtime	containerd (default) vs gVisor	containerd for control plane, gVisor for untrusted workloads	Security Lead
8	Observability vendor	OpenSearch (chosen) vs Grafana Cloud	Self-hosted OpenSearch + Grafana	SRE Lead
9	EU AI inference	SageMaker (Mistral) vs Bedrock (Claude)	SageMaker for true EU residency	Compliance Lead
10	On-prem deployment model	EKS Anywhere vs K8s-manual	EKS Anywhere for repeatability	Eng Lead

Closing Notes

This document is the **AWS deployment target** for Plumb. It complements:

- PLUMB-PHASE-0-REPORT.md — what was built in Phase 0 (the foundation)
- ~/.hermes/plans/2026-08-08_090307-plumb-visual-intelligence-platform.md — the master plan produced by the 10-agent planning pass
- ~/.hermes/plans/2026-08-08_090307-plumb-visual-intelligence-platform-APPENDIX.md — the technical appendix with resolved cross-agent conflicts

The full system spans 13 products, ~\$8.4M Year-1 budget, 12 founding hires, multi-region AWS deployment, FedRAMP Moderate authorization, 14 vendor integrations, and native mobile apps. This document specifies how it all runs on AWS. Phase 0 has shipped the foundation; the gap between this document and the working code is the execution roadmap.

End of architecture document.